Adaptive RDMA Optimization for Distributed LLM Workloads

A Consolidated Technical Report

Nathan Gopee
MS Computer Science Candidate
SUNY New Paltz
gopeen1@newpaltz.edu

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

February 2026

Abstract

Project Purpose. When you run a large language model across multiple GPUs, those GPUs spend 30–60% of their cycles waiting for data to arrive over the network. Existing communication libraries (NCCL, Gloo, UCX) route all tensor traffic through the same static channels, whether it is a small latency-sensitive KV cache page or a large bulk gradient synchronization. That creates *head-of-line blocking*—big transfers hold up the small urgent ones, like a highway where sports cars and semi-trucks share a single lane. On top of that, different GPUs use different precision formats: V100s operate in FP16 while newer GPUs like the RTX 5070 Ti use BF16—same 16 bits, different exponent/mantissa trade-off. Current tools do not reconcile these mismatches on the fly, so mixed clusters waste cycles on manual conversion and risk silent precision drift.

This project develops an adaptive RDMA middleware that addresses both problems. A Weighted Finite Automaton (WFA) classifier monitors tensor access patterns at runtime and categorizes each stream as hot, warm, or cold. Hot traffic (KV cache, attention scores) routes through dedicated low-latency Queue Pairs; cold traffic (weights, gradients) batches into throughput-optimized channels overlapped with computation. A centralized RDMA Tensor Cache in the Neumann runtime maintains FP32 master copies of shared tensors and applies stochastic rounding for precision conversion across heterogeneous GPU boundaries. The middleware supports pluggable transports (TCP, Soft-RoCE, hardware RoCEv2, Tesla TTPoe) and targets <5% precision-switching overhead, >90% prefetch hit rates, and 4× bandwidth reduction via MXFP4 quantization—reducing communication-induced stalls by an order of magnitude over default NCCL on 100 GbE fabrics.

Contents

1 Foundations of Natural Language Processing **5**
- 1.1 Word Representations . . . 5
- 1.2 Sequence-to-Sequence and the Attention Mechanism . . . 5
- 1.3 Vectors, Weights, and Tensors . . . 6

2 The Transformer Architecture **6**
- 2.1 Attention Is All You Need . . . 6
- 2.2 FlashAttention Evolution . . . 8
- 2.3 PagedAttention . . . 9
- 2.4 GPU Architecture Evolution: From Graphics to Inference . . . 9
  - 2.4.1 Volta: 1st Generation Tensor Cores (2017) . . . 9
  - 2.4.2 Turing and Ampere: Expanding Precision Support . . . 10
  - 2.4.3 Hopper: Asynchronous Everything . . . 10
  - 2.4.4 Blackwell: Tensor Memory and Cross-SM Fusion . . . 11
  - 2.4.5 Implications for RDMA Tensor Transfers . . . 11

3 Scheduling, Scaling, and Expert Parallelism **12**
- 3.1 Scheduling Algorithms for Tensor Transfers . . . 12
  - 3.1.1 Classical Optimization Algorithms . . . 13
  - 3.1.2 Queueing-Theoretic Approaches . . . 13
  - 3.1.3 Learning-Based Adaptive Algorithms . . . 14
  - 3.1.4 Comparative Analysis and Applicability . . . 15
  - 3.1.5 Mapping to RDMA Tensor Scheduling . . . 15
- 3.2 Mixture of Experts . . . 16
- 3.3 Scaling Laws and the Communication Bottleneck . . . 17

4 RDMA Fundamentals and the Communication Bottleneck **17**
- 4.1 RDMA Programming Model . . . 17
- 4.2 The Communication Bottleneck . . . 18
- 4.3 Transport Technologies . . . 18

5 Problem Statement and Contributions **19**
- 5.1 The Multi-Standard Fragmentation Crisis . . . 19
- 5.2 Research Objectives . . . 19
- 5.3 What vLLM Already Provides . . . 20

6 Critical Issues Analysis **20**
6.1 Issue 1: cudaMallocAsync IPC Incompatibility (UCX #7110) . . . 20
6.2 Issue 2: Transport Selection Robustness (UCX #9908) . . . 21
6.3 Issue 3: Container Shared Memory Isolation (UCX #10055) . . . 21
6.4 Issue 4: Cross-NUMA GPU Latency Regression (UCX #3249) . . . 22
6.5 Issue 5: RoCE Reliability Timeouts (UCX #6000) . . . 22
6.6 Issue 6: Heterogeneous Cluster Builds (UCX #10273) . . . 22
6.7 Issue 7: NCCL-UCX Container Integration (UCX #10055) . . . 22

7 Hardware Platforms **23**
7.1 Phase 1: SUNY New Paltz Lab Cluster . . . 23
7.2 Phase 2: Two-Tower Home Lab . . . 23
7.3 Transition Rationale . . . 23
7.4 Software Stack . . . 24

8 System Architecture and Design **24**
8.1 Architecture Overview . . . 24
8.2 WFA Tensor Classifier . . . 24
8.3 Smart RDMA Routing . . . 26
8.4 Phase-Aware Classification . . . 27
8.5 KV Cache Placement and Migration . . . 27
8.6 Joint Training-Inference Scheduling . . . 27
8.7 Transport Middleware . . . 27
8.8 EXO Integration . . . 28
8.9 vLLM Integration . . . 28

9 Security Considerations **29**

10 RDMA Tensor Cache and Precision Management **29**
10.1 The Precision Mismatch Problem . . . 29
10.2 Precision Pipeline . . . 29
10.3 Stochastic Rounding . . . 30
10.4 Neumann Integration . . . 30
10.5 Centralized KV Cache . . . 30

11 Benchmarking and Validation **31**
11.1 Soft-RoCE Baseline . . . 31
11.2 Tesla TTPoe Build . . . 32
11.3 FlashAttention 3 Blackwell Fix . . . 32
11.4 gpt-oss-120b TCP vs RDMA . . . 32
11.5 SoftRoCEv2 Two-Tower Baseline . . . 33

11.6 Parallelism Analysis . . . 33

12 Fine-Tuning and SAE Feature Steering **34**
12.1 Heterogeneous Training Pipeline . . . 34
12.2 Sparse Auto-Encoder Feature Steering . . . 34
12.3 SAE Steering Validation . . . 35

13 Experimental Design **36**
13.1 Three Pillars . . . 36
13.2 UET-HTSIM Simulation . . . 36
13.3 Performance Targets . . . 37
13.4 Ablation Studies . . . 37

14 Implementation Roadmap: Project Breakdown **37**
14.1 Completed Projects . . . 37
14.2 Remaining Projects . . . 38
14.3 Project Dependencies . . . 38

15 Future Work and Conclusions **38**
15.1 UCX/UCC Integration Path . . . 38
15.2 RISC-V Assembly Generation . . . 38
15.3 Conclusions . . . 38

A Scripts and Configurations **43**

B Expected Transport Performance **43**

1 Foundations of Natural Language Processing

The objects transferred between GPUs during distributed LLM training and inference—embedding vectors, weight matrices, gradient tensors, activation tensors, and KV cache entries—are all tensors. Understanding how these tensors arise from the evolution of NLP architectures motivates the design of an RDMA middleware that treats them as first-class citizens with distinct routing requirements.

1.1 Word Representations

Early NLP systems represented words as one-hot vectors: a vocabulary of V words produced V-dimensional binary vectors with a single nonzero entry. This encoding contains no semantic information—the distance between any two words is identical—and produces sparse, high-dimensional tensors that waste both memory and bandwidth.

Mikolov et al. [23] introduced Word2Vec, which learns dense, low-dimensional embeddings by training a shallow neural network to predict context words. Each word maps to a vector in $\mathbb{R}^d$ (typically $d = 300$), where geometric proximity encodes semantic similarity. The embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$ is the first weight tensor that must be distributed across GPUs when V is large.

Pennington et al. [34] extended this with GloVe, which combines the advantages of global matrix factorization (like LSA) with local context windows (like Word2Vec). GloVe embeddings are trained on word co-occurrence statistics from a corpus, producing vectors where vector differences encode semantic relationships: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$.

These embedding vectors are the simplest tensors in an LLM: rank-1 objects of shape $(d,)$ for a single word, or rank-2 matrices of shape (V, d) for the full vocabulary table. Under tensor parallelism, the embedding table is typically sharded column-wise across GPUs, with each GPU storing $V \times (d/\text{TP})$ elements.

1.2 Sequence-to-Sequence and the Attention Mechanism

Sutskever et al. [43] formalized the encoder-decoder architecture for sequence-to-sequence tasks. An encoder RNN processes the input sequence into a fixed-length context vector $\mathbf{c}$, and a decoder RNN generates the output sequence conditioned on $\mathbf{c}$. The information bottleneck is immediate: compressing an arbitrarily long input into a single vector $\mathbf{c} \in \mathbb{R}^h$ loses information for long sequences.

Bahdanau et al. [4] resolved this bottleneck with the attention mechanism. Instead of compressing the entire input into one vector, the decoder attends to all encoder hidden states at each output step:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \qquad \mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j \tag{1}$$

where $e_{ij} = a(\mathbf{s}_{i-1}, \mathbf{h}_j)$ is a learned alignment function, $\mathbf{s}_{i-1}$ is the decoder state, and $\mathbf{h}_j$ are encoder hidden states.

This mechanism generates the attention score tensors that dominate KV cache memory in

modern LLMs. It also introduces the fundamental parallelism limitation of RNNs: because each hidden state $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, the encoder cannot be parallelized across the sequence dimension. For a 2048-token input, the encoder must execute 2048 sequential steps regardless of available compute—a fatal inefficiency for GPU clusters that excel at parallel computation.

1.3 Vectors, Weights, and Tensors

To understand what RDMA transfers between GPUs, we define the tensor types that arise in transformer-based LLMs:

- **Embedding vectors** ($V \times d_{\text{model}}$): Map discrete tokens to continuous representations. Sharded across GPUs under tensor parallelism.
- **Weight matrices** (W_Q, W_K, W_V, W_O of shape $d_{\text{model}} \times d_{\text{head}} \times n_{\text{heads}}$; MLP weights of shape $d_{\text{model}} \times d_{\text{ff}}$): The learned parameters. Stored in FP32 for master copies, cast to FP16/BF16 for compute.
- **Gradient tensors**: Same shape as their corresponding weights. Produced during backpropagation, accumulated across data-parallel replicas via AllReduce.
- **Activation tensors** ($b \times s \times d_{\text{model}}$): Intermediate results between transformer layers. Transferred at pipeline stage boundaries.
- **KV cache tensors** ($b \times n_{\text{heads}} \times s \times d_{\text{head}}$): Cached key and value projections from prior tokens, growing linearly with sequence length during autoregressive generation.

Tensor *rank* (order) ranges from 0 (scalar loss value) through 1 (bias vector), 2 (weight matrix), 3 (per-layer KV cache), to 4 (batched multi-head attention output). Each rank implies different memory layouts, access patterns, and optimal transfer strategies. The RDMA Tensor Cache treats these as first-class objects with metadata (precision format, access frequency, prefetch priority) that drives routing decisions.

2 The Transformer Architecture

2.1 Attention Is All You Need

Vaswani et al. [48] replaced recurrence entirely with self-attention, enabling full parallelization across the sequence dimension. The architecture processes input in three stages: tokenization, embedding, and contextual refinement through attention.

Tokens and Embeddings. Natural language input is first broken into tokens—words, subwords, or punctuation marks. Each token maps to a dense embedding vector via a lookup table, producing a matrix $X \in \mathbb{R}^{s \times d}$ where s is the sequence length and $d = d_{\text{model}}$ is the embedding dimension. At this stage, each token consistently maps to the same vector regardless of context. Positional

encodings are added to inject sequence order, since self-attention is permutation-invariant. The original transformer uses sinusoidal encodings; modern LLMs replace these with Rotary Position Embeddings (RoPE) or learned position embeddings.

Query, Key, and Value Projections. The attention block enables embedding vectors to exchange information and update their meanings based on context. Three learned weight matrices—W_Q, W_K, W_V—project each token embedding into *query*, *key*, and *value* vectors. Intuitively, the query matrix allows each word to "ask a question" about what context it needs (e.g., a noun looking for a modifying adjective). The key matrix enables other words to "respond" to these queries, indicating their relevance. The dot product between a query and all keys produces an *attention pattern* that reveals which words are relevant to updating each other's meaning. The softmax function normalizes these dot products into weights summing to one. Finally, the value matrix produces vectors that, when combined via these attention weights, update each token's embedding with the relevant contextual information.

Formally, scaled dot-product attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2}$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and the $\sqrt{d_k}$ scaling prevents dot products from growing large enough to push softmax into regions with vanishingly small gradients. The output is a weighted sum of value vectors along the columns of the attention pattern—each sum represents the contextual update to be added to the original token's embedding, producing a more contextually enriched representation.

Multi-Head Attention. Rather than computing a single attention function, multi-head attention runs h parallel attention functions, each with independent learned projections:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O \tag{3}$$

where $\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$ and $W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is the output projection. Each head can attend to different aspects of context—one head might track syntactic relationships while another captures semantic similarity. This is why the model can simultaneously disambiguate a word like "mole" (the animal vs. the unit of measurement) while also encoding the broader context of the surrounding sentence.

A transformer encoder block consists of multi-head self-attention followed by a position-wise feed-forward network (FFN), each with residual connections and layer normalization:

$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{MultiHead}(\mathbf{x})), \quad \mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z})) \tag{4}$$

For distributed inference, the critical observation is that each transformer layer produces activation tensors of shape (b, s, d_{model}) that must be communicated between pipeline stages, and KV cache tensors of shape (b, h, s, d_k) per layer that accumulate during generation.

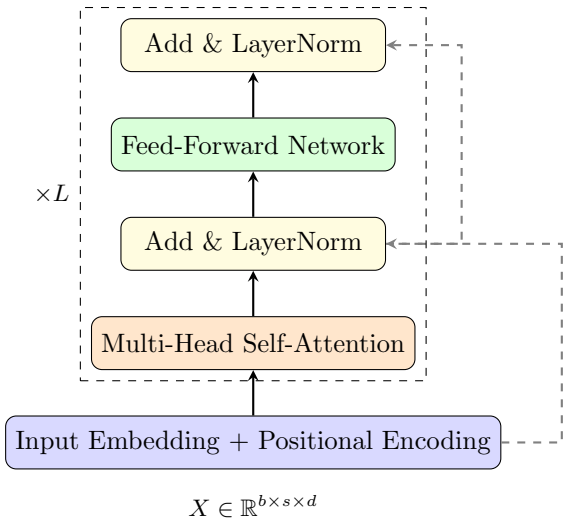

Figure 1: Transformer encoder block. The block repeats L times. Under tensor parallelism, the multi-head attention and FFN layers are sharded across GPUs, requiring AllReduce communication after each.

2.2 FlashAttention Evolution

The standard attention computation has $O(s^2)$ memory complexity due to materializing the $s \times s$ attention matrix. For long sequences ($s = 8192$ or higher), this becomes the dominant memory bottleneck.

FlashAttention [8] introduced IO-aware tiling that computes attention in blocks, keeping the working set in GPU SRAM rather than HBM. By avoiding the materialization of the full attention matrix, FlashAttention achieves exact attention with $O(s)$ memory and 2–4× wall-clock speedup on GPT-2 training.

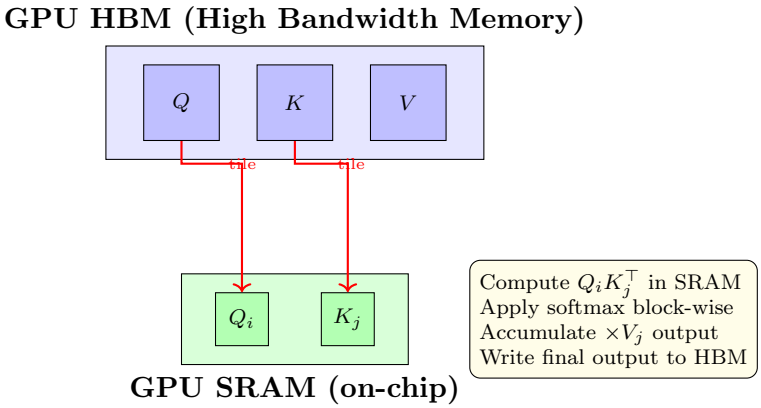

Figure 2: FlashAttention tiling. Instead of materializing the full $s \times s$ attention matrix in HBM, FlashAttention loads tiles of Q and K into SRAM, computes partial attention, and accumulates results incrementally.

FlashAttention-2 [7] improved parallelism by partitioning work across warps within a thread

block, achieving 50–73% of peak FLOPs on A100 GPUs. The key insight is reducing synchronization between warps by assigning non-overlapping rows of the output matrix to different warps.

FlashAttention-3 [41] targets Hopper (H100) and Blackwell architectures, exploiting hardware asynchrony between tensor cores, memory loads, and softmax computation. It introduces FP8 quantization for the $QK^\top$ computation, achieving 740 TFLOPs/s on H100—1.5–2× faster than FA2. Section 11.3 documents our fix enabling FA3 on Blackwell (RTX 5090) GPUs.

2.3 PagedAttention

Kwon et al. [15] observed that KV cache memory in serving systems is severely fragmented: each request reserves contiguous memory for its maximum possible sequence length, wasting 60–80% of allocated KV memory. PagedAttention applies the operating system concept of virtual memory paging to KV cache management.

KV cache is divided into fixed-size blocks (e.g., 16 tokens per block). A block table maps logical token positions to physical memory blocks, enabling non-contiguous storage. This achieves near-zero waste and enables memory sharing across requests with common prefixes.

PagedAttention's block-based KV cache is precisely what our RDMA middleware transfers between GPUs. The block granularity (typically 16–64 tokens $\times$ d_{head} $\times$ 2 bytes = 2–16 KB per block) maps well to RDMA transfer sizes where the NIC can achieve high throughput without excessive protocol overhead.

2.4 GPU Architecture Evolution: From Graphics to Inference

Early NVIDIA GPUs were designed for rasterization and shading—workloads dominated by independent per-pixel computations. These architectures excelled at vision tasks (image classification, object detection) because convolutional neural networks exhibit the same per-element parallelism: each pixel's convolution kernel operates independently, mapping naturally onto thousands of CUDA cores executing identical instructions across different data (SIMT). The bottleneck was compute throughput, not data movement.

Modern LLM inference inverts this relationship. Autoregressive token generation is inherently sequential (each token depends on the previous token's KV cache), the attention mechanism requires dense matrix-matrix multiplications across the entire context window, and the dominant cost shifts from FLOPs to memory bandwidth [32]. This architectural mismatch drove NVIDIA to redesign GPU hardware from the transistor level upward, evolving Tensor Cores across five generations to match the compute patterns of transformer workloads.

2.4.1 Volta: 1st Generation Tensor Cores (2017)

Volta (V100) introduced the first Tensor Cores, purpose-built Matrix-Multiply-Accumulate (MMA) units that operate on FP16 inputs with FP32 accumulation [32]. Each Tensor Core computes a $4 \times 4 \times 4$ fused multiply-add per cycle. The MMA instruction is *warp-scoped*: a warp of 32 threads

collectively feeds an $8 \times 8 \times 4$ matrix tile (shape `m8n8k4`) to the Tensor Cores, producing 1,024 FP16 FLOPs per cycle per SM.

Volta's Tensor Cores delivered an order-of-magnitude improvement over CUDA cores for matrix operations, but they support only FP16 input $\rightarrow$ FP32 accumulate. Our V100 GPUs in Tower 2 inherit this limitation: tensors must be in IEEE FP16 format before Tensor Core execution. BF16 is not supported in hardware, which creates the precision mismatch with the RTX 5070 Ti (BF16-native) that motivates the RDMA Tensor Cache's stochastic rounding pipeline (Section 10.4).

2.4.2 Turing and Ampere: Expanding Precision Support

Turing (2nd generation, 2018) added INT8 and INT4 Tensor Core paths, enabling efficient inference quantization. Ampere (3rd generation, A100, 2020) expanded further with BF16 support and `ldmatrix`—an instruction that loads a warp-wide matrix tile from shared memory into registers in a single operation. Ampere also introduced *asynchronous data copies* (`cp.async`), decoupling global-to-shared memory transfers from compute [32]. The MMA scope widened from warp-scoped to *warp-level synchronous*: the `mma.sync` instruction operates at the warp level with shape `m16n8k16`, doubling throughput to 2,048 FP16 FLOPs/cycle/SM. Ampere also introduced *2:4 structured sparsity*, which prunes 2 of every 4 weight elements, theoretically doubling Tensor Core throughput by halving the work.

2.4.3 Hopper: Asynchronous Everything

Hopper (H100, 4th generation, 2022) fundamentally restructured the GPU memory hierarchy for LLM workloads with three innovations [32]:

1. **Thread Block Clusters:** A new hierarchy level where multiple CTAs (Cooperative Thread Arrays) across different SMs form a cluster. CTAs within a cluster can access each other's shared memory via *Distributed Shared Memory* (DSMEM), enabling cross-SM data sharing without going through L2 cache or HBM.

2. **Tensor Memory Accelerator (TMA):** A hardware unit that handles multi-dimensional tensor addressing and data movement between global memory and shared memory. TMA supports *multicast loads*: a single instruction loads data from global memory into the shared memory of multiple SMs simultaneously, reducing L2 cache traffic and HBM bandwidth consumption. However, TMA has higher latency than regular `cp.async` for small transfers (due to address generation overhead), making it best suited for large, regular data movements rather than small KV cache page loads [32].

3. **Warpgroup-level Asynchronous MMA (`wgmma`):** The MMA scope expands again—a warpgroup of 4 warps (128 threads) collectively performs one MMA operation. `wgmma` reads operand A from registers or shared memory and operand B exclusively from shared memory, reducing register pressure. The shape expands to `m64nNk16` where N ranges from 8 to 256,

yielding 4,096 FP16 FLOPs/cycle/SM. Critically, Hopper introduces FP8 (E4M3 and E5M2) Tensor Core paths with FP32 accumulation—though the accumulation path is actually a 22-bit fixed-point format (13-bit mantissa), requiring periodic writeback to CUDA cores for full-precision accumulation [32].

FlashAttention-3 [41] exploits all three Hopper features: TMA for asynchronous $Q/K/V$ tile loads, `wgmma` for the attention matrix multiply, and DSMEM for cross-SM K/V sharing during multi-query attention. This achieves 740 TFLOPs/s (1.5–2× over FA2)—demonstrating that the memory system is now the primary bottleneck, not the Tensor Cores themselves.

2.4.4 Blackwell: Tensor Memory and Cross-SM Fusion

Blackwell (GB100/GB200, 5th generation, 2024) targets inference-at-scale with two architectural departures [32]:

1. **Tensor Memory (TMEM):** A new 256 KB memory per SM, dedicated to Tensor Core operands. TMEM has 128 rows (lanes) × 512 columns of 4-byte cells, with a restricted access pattern: a warpgroup of 4 warps collectively accesses the full TMEM, each warp owning 32 lanes. By limiting access patterns, NVIDIA reduces access port count and chip area. Unlike shared memory, TMEM must be explicitly managed by the programmer (allocation, deallocation, copy-in/copy-out).

2. **5th Generation MMA (`tcgen05.mma`):** The MMA instruction moves to *single-thread semantics*—a single thread issues the MMA, reading operand A and writing output D to TMEM, while reading operand B from shared memory. This eliminates the complex register-to-Tensor-Core data layouts of previous generations and frees thread registers for other work (epilogue operations like activation functions). A notable variant, `MMA.2SM`, fuses two SMs in a CTA pair (mapped to a Texture Processing Cluster) to collectively perform a single larger MMA, doubling the M dimension to `m256nNk16` and halving the B-matrix loads via DSMEM sharing between the paired SMs.

Blackwell doubles Tensor Core throughput to 8,192 FP16 FLOPs/cycle/SM across 148 SMs, with 186 GB HBM3e at 8,000 GB/s. It adds native MXFP8, MXFP6, and MXFP4 microscaling format support [40], plus NVIDIA's proprietary NVFP4 format which uses smaller block sizes and two-level quantization for higher accuracy than standard MXFP4.

2.4.5 Implications for RDMA Tensor Transfers

The five-generation evolution reveals a consistent trend: NVIDIA scales Tensor Core size (FLOPs per SM) faster than the number of Tensor Cores, because matrix multiplication arithmetic intensity scales cubically ($O(n^3)$ FLOPs) while data movement scales quadratically ($O(n^2)$ reads) [32]. Larger tiles amortize memory latency better. But this means the gap between compute capability and

Table 1: Tensor Core architecture evolution across GPU generations [32]. Each generation shifts the bottleneck further from compute to memory bandwidth, motivating RDMA optimization.

	Volta (V100)	**Ampere (A100)**	**Hopper (H100)**	**Blackwell (GB200)**
Generation	1st	3rd	4th	5th
SMs	80	108	132	148
FP16 FLOPs/cycle/SM	1,024	2,048	4,096	8,192
HBM Capacity	16 GB	40 GB	80 GB	186 GB
HBM Bandwidth	900 GB/s	1,555 GB/s	3,350 GB/s	8,000 GB/s
L2 Cache	6 MB	40 MB	50 MB	130 MB
Shared Mem / SM	96 KB	164 KB	228 KB	228 KB
MMA Scope	Warp	Warp	Warpgroup	Single Thread
FP16 MMA Shape	`m8n8k4`	`m16n8k16`	`m64nNk16`	`m256nNk16`
MMA SASS	HMMA	HMMA	HGMMA	UTCHMMA
Lowest Precision	FP16	INT4	FP8 (E4M3)	NVFP4 / MXFP4

memory bandwidth widens each generation—the GPU can process tensors faster than it can fetch them from HBM, let alone receive them over a network.

For our heterogeneous cluster, this has three direct consequences:

1. **Precision diversity:** The V100 supports only FP16 Tensor Core input; the 5070 Ti (Blackwell-class consumer) supports BF16 and FP8. Every tensor transferred between towers must undergo precision conversion. The RDMA Tensor Cache's FP32 master weights and stochastic rounding pipeline exist precisely because these GPUs cannot share a native precision format.

2. **Memory hierarchy depth:** Hopper and Blackwell add DSMEM and TMEM respectively, creating additional levels that tensor data must traverse. RDMA-delivered tensors land in HBM via GPUDirect; efficient kernels must then tile them through L2 $\rightarrow$ shared memory $\rightarrow$ TMEM/registers before Tensor Cores can consume them. The prefetch FSM (Section 10.4) must account for this multi-level pipeline.

3. **Transfer size alignment:** Blackwell's `m256nNk16` tiles are 16$\times$ larger than Volta's `m8n8k4`. RDMA transfer sizes should align with the destination GPU's tile granularity to avoid partial-tile stalls. The WFA classifier's hot/cold categorization implicitly captures this: small, latency-sensitive transfers (KV cache pages matching Volta's smaller tiles) take the hot path, while large weight tiles (matching Blackwell's larger MMA shapes) take the cold path with batching.

3 Scheduling, Scaling, and Expert Parallelism

3.1 Scheduling Algorithms for Tensor Transfers

Efficient scheduling of tensor transfers across an RDMA fabric requires algorithms that overlap communication with computation and respect data dependencies. The problem is structurally analogous to GPU cluster scheduling, where jobs with heterogeneous resource demands and interdependencies

must be assigned to accelerators with minimal idle time. Chab et al. [5] provide a comprehensive taxonomy of GPU scheduling algorithms organized into four algorithmic families—classical optimization, queueing-theoretic, learning-based, and hybrid—each with distinct applicability to tensor transfer scheduling.

3.1.1 Classical Optimization Algorithms

Classical schedulers rely on deterministic rules or mathematical optimization without any learning component [5]. The simplest are *greedy algorithms*: Shortest-Job-First (SJF) processes the job with the smallest expected runtime next, minimizing average completion time on a single GPU. For tensor transfers, this maps to prioritizing small, latency-sensitive tensors (KV cache pages, attention scores) over large bulk transfers (gradient accumulations)—the same intuition behind our hot/cold path differentiation. However, SJF can starve large transfers indefinitely; production systems mitigate this with *aging* policies that promote long-waiting transfers to higher priority [12].

Backfilling extends FCFS by allowing smaller transfers to leapfrog the head-of-queue when resources are idle, while reserving capacity for the blocked head transfer. In RDMA terms, small KV cache reads can proceed on available Queue Pairs while a large gradient AllReduce waits for all QPs in its group. First-Fit (FF) and Best-Fit (BF) packing heuristics reduce multi-GPU placement to a bin-packing problem; on *heterogeneous* clusters like ours (V100 + 5070 Ti), Gavel [24] normalizes job throughput across GPU types by scheduling in the transformed space of *effective GPU-seconds*, extending classical packing heuristics to handle device heterogeneity while preserving fairness.

Dynamic programming (DP) computes exact schedules for small problem instances. Production GPU schedulers often embed DP selectively: OASiS uses polynomial-time DP inside an online primal-dual algorithm to determine high-priority worker/parameter-server allocations, achieving near-optimal throughput with per-decision latencies below 100 ms [5]. For RDMA tensor scheduling, a lightweight DP could optimize the assignment of tensor batches to QPs within a single scheduling epoch.

ILP/MILP formulations provide provably optimal schedules but are computationally expensive (NP-hard even for $m = 2$ GPUs [5]). MILP is practical for offline calibration—computing optimal QP assignments for a fixed model architecture—then deploying the resulting allocation table at runtime.

Meta-heuristics (genetic algorithms, simulated annealing, Tabu search) explore large search spaces but require seconds to minutes per schedule, making them unsuitable for online tensor scheduling. They remain useful for offline parameter tuning: running a GA on historical traces to optimize scheduling-policy weights, then deploying the tuned heuristic for real-time decisions.

3.1.2 Queueing-Theoretic Approaches

Queueing-theoretic schedulers assign each tensor transfer a dynamic priority based on its service history, rather than solving a global optimization [5]. *Index policies* compute a per-transfer score that balances fairness and efficiency. The Gittins index is a canonical example for $M/G/1$ queues:

a transfer with attained service v receives a priority score that rewards transfers likely to complete soon. While exact computation is complex, approximations yield near-optimal mean-slowdown performance when transfer sizes are unknown.

Least Attained Service (LAS) always runs the transfer with the smallest accumulated service time, achieving at most 25% penalty over SRPT for mean slowdown under heavy-tailed distributions [5]. Tiresias [12] adapts LAS to GPU clusters using two-dimensional LAS (2D-LAS) that tracks both time and GPU count, preempting transfers with fixed-length leases rather than costly full preemption. E-LAS extends this by incorporating real-time epoch progress rates. For RDMA tensor transfers, a LAS-inspired policy would track cumulative bytes transferred per tensor stream and prioritize streams with the least accumulated transfer volume—naturally favoring small, infrequent transfers (KV cache) over bulk data (weights).

Multi-class priority queues divide transfers into priority classes—"hot" for latency-critical KV cache and attention tensors, "warm" for pipeline activations, "cold" for bulk gradient and weight transfers. Strict priority in an $M/M/1$ system guarantees minimal response for the hot class, though cold transfers may experience increased delays [5]. Our WFA classifier (Section 8) implements exactly this pattern, with learned transition weights that adapt class boundaries to workload phase (prefill vs. decode).

Predictive admission control uses queueing theory formulas to drive service-level guarantees. Little's Law ($L = \lambda W$), mean response-time bounds, and $M/M/1$ approximations can trigger autoscaling of RDMA QPs or more aggressive preemption when congestion looms—directly relevant to our RoCE timeout mitigation (Section 6).

3.1.3 Learning-Based Adaptive Algorithms

Learning-based schedulers harness machine learning to adapt scheduling policies from data [5]. *Supervised prediction models* are trained on job features to forecast metrics like runtime, memory footprint, or scaling behavior. A scheduler then implements data-driven SJF by ordering transfers according to predicted completion times. In heterogeneous GPU clusters, Gavel [24] benchmarks workloads to derive throughput estimates for different GPU types, adapting classical policies to mixed environments.

Reinforcement learning (RL) casts scheduling as a sequential decision problem. DeepRM [20] encodes the cluster as an image of resource occupancy and trains a neural network to output scheduling distributions. Decima [21] extends this with graph neural networks that embed DAG-structured workloads, outperforming heuristics like SRTF under variable load. HeterPS applies LSTM-based RL to allocate tasks across CPUs and GPUs in heterogeneous clusters [5].

RL faces four practical challenges for production deployment [5]: (1) *exploration vs. exploitation*—suboptimal actions during training can degrade live performance; (2) *state-space explosion*—real clusters with hundreds of concurrent transfers exceed tractable state representations; (3) *stability and safety*—unconstrained RL policies may starve critical transfers or violate SLOs; and (4) *interpretability*—opaque policies hinder debugging in mission-critical environments. Hybrid designs

address these by combining learned models for parameter tuning with proven rule-based control logic for the primary decision path.

Adaptive scheduling via feedback control continuously monitors system metrics and adjusts scheduling behavior in a closed loop [5]. Pollux [36] uses online goodput feedback per job to dynamically adjust GPU allocations: if additional GPUs do not improve goodput (diminishing returns), the system reduces that job's share and reallocates to others. AntMan similarly modifies deep learning frameworks for fine-grained, real-time scaling of GPU memory and compute without requiring job restarts. For RDMA tensor scheduling, a feedback-driven approach would monitor transfer completion rates, QP queue depths, and congestion signals (ECN marks, RTT variance), then adjust hot/cold thresholds and QP allocation dynamically.

3.1.4 Comparative Analysis and Applicability

Chab et al. [5] evaluate scheduling algorithm families across four dimensions. We summarize the key trade-offs relevant to tensor transfer scheduling:

Table 2: Comparative analysis of GPU scheduling algorithm families [5] and their applicability to RDMA tensor scheduling

Family	Decision Speed	Schedule Quality	Heterogeneity	RDMA Tensor Application
Greedy / Heuristic	$O(\log n)$, sub-ms	Good avg., no worst-case bounds	Extensible (Gavel)	Hot/cold QP assignment, backfill for small KV transfers
ILP / MILP	Seconds–minutes	Provably optimal	Via constraints	Offline QP allocation calibration
Queueing-Theoretic	$O(1)$ per-job index	Near-optimal fairness (Gittins)	Per-resource queues	LAS-based transfer prioritization, admission control
Learning-Based (RL)	ms (inference)	Near-optimal (trained)	Encoded in state	Adaptive threshold tuning, congestion prediction
Hybrid	ms (heuristic) + periodic optimization	Strong practical	Flexible	WFA classifier + periodic QP rebalancing

The survey identifies that no single paradigm dominates across all scenarios. *Offline deterministic* workloads favor ILP/DP for optimal schedules. *Online workloads with heavy-tailed transfer sizes*—typical of LLM inference where KV cache transfers are small and frequent while gradient syncs are large and periodic—benefit from queueing-theoretic policies like LAS or Gittins that balance fairness and mean response time. *Heterogeneous resources* (our V100+5070 Ti cluster) require two-stage approaches: first matching transfers to resource classes, then applying standard scheduling within each class [24]. *Multi-tenant cloud environments* demand fairness mechanisms; Themis [19] uses partial-auction frameworks supplemented by SJF inner loops.

Production systems converge on *hybrid strategies*: lightweight heuristics for real-time dispatch, ILP or matching solvers for small-scale assignment tasks, and learning models for runtime and resource-usage prediction [5]. Our middleware adopts this hybrid pattern: the WFA classifier provides the heuristic real-time dispatch (hot/warm/cold in $O(1)$), periodic telemetry analysis provides the optimization feedback loop, and the prefetch engine's pattern classifier (Section 10.4) provides the learned prediction component.

3.1.5 Mapping to RDMA Tensor Scheduling

We additionally draw analogies from quantum circuit scheduling [52], where three algorithmic patterns map to RDMA tensor transfer optimization:

Table 3: Scheduling algorithm mapping from quantum/GPU domains to RDMA tensor transfers

Scheduling Concept	Original Domain	RDMA Tensor Application	WFA State
BBOP (Batch-Buffered Overlap)	Quantum circuit gate batching	Batch cold-path tensor transfers; overlap with hot-path RDMA writes	Cold → Batched
SMGP (Staggered Multi-Gate Parallelism)	Parallel quantum gate execution	Pipeline-parallel activation overlap; stagger layer transfers across QPs	Warm → Pipelined
DAGC (Dependency-Aware Gate Contraction)	Circuit DAG optimization	WFA-driven tensor coalescing; merge adjacent cold tensors into single RDMA write	Cold → Coalesced
Priority-based scheduling [5]	GPU job scheduling	Hot/cold path differentiation; dedicated QPs for latency-critical tensors	Hot → Priority QP
Preemptive scheduling [5]	OS process scheduling	Tensor reclassification during phase changes (prefill → decode)	State transition
Feedback-driven scaling [36]	GPU goodput optimization	Monitor QP queue depth; adjust hot/cold thresholds via telemetry	Adaptive → Rebalance

The BBOP pattern is particularly relevant: during training, gradient tensors from completed layers can be batched into a single RDMA transfer while the next layer's forward pass proceeds. The prefetch engine (Section 10.4) implements a variant of SMGP by issuing speculative RDMA reads for upcoming layers while the current layer computes. The feedback-driven approach from Pollux [36] informs our telemetry-based QP rebalancing: monitoring transfer completion rates per QP and reallocating capacity from underutilized cold-path channels to congested hot-path channels.

The survey's analysis of *practical integration challenges*—resource-manager interfacing, distributed workload coordination, framework coupling, GPU-specific APIs, scheduler overhead, preemption cost, and monitoring overhead [5]—directly applies to our middleware. Our transport abstraction layer addresses framework coupling by providing a stable interface across vLLM, exo, and custom training loops. The container-aware communication broker (Section 6) addresses the deployment challenges that Chab et al. identify as a gap between algorithmic elegance and engineering practicality.

3.2 Mixture of Experts

Mixture-of-Experts (MoE) models use conditional computation to increase model capacity without proportionally increasing compute. GShard [16] introduced expert parallelism for distributed MoE training, where experts are distributed across GPUs and a gating network routes each token to its top-k experts.

The Switch Transformer [11] simplified this to top-1 routing: each token is sent to exactly one expert, reducing the communication overhead from $O(k \cdot E)$ to $O(E)$ AllToAll messages. With E experts distributed across N GPUs, each GPU hosts E/N experts and must exchange tokens with all other GPUs via AllToAll.

Expert parallelism interacts with tensor and data parallelism through:

$$\texttt{EP_SIZE} = \texttt{TP} \times \texttt{DP} \tag{5}$$

as documented in the AMD ROCm guide [2]. The AllToAll communication pattern is inherently asymmetric—different tokens route to different experts, creating variable-size messages—which makes it particularly amenable to adaptive routing. Hot experts (those receiving many tokens) generate larger, more frequent transfers that benefit from priority QP assignment.

3.3 Scaling Laws and the Communication Bottleneck

Kaplan et al. [14] established power-law relationships between model size, dataset size, compute budget, and loss. For a model with N parameters trained on D tokens with compute budget C:

$$L(N) \propto N^{-0.076}, \quad L(D) \propto D^{-0.095}, \quad L(C) \propto C^{-0.050} \tag{6}$$

Hoffmann et al. [13] refined this with the Chinchilla scaling law: for compute-optimal training, a model with N parameters should be trained on approximately $20N$ tokens. This implies that as models grow from 7B to 70B to 700B parameters, training data scales proportionally, and the communication overhead of distributing both model and data across GPUs grows superlinearly.

For a model distributed across P GPUs with tensor parallelism degree T, the per-layer AllReduce volume is:

$$V_{\text{AllReduce}} = 2 \cdot b \cdot s \cdot d \cdot \frac{T-1}{T} \tag{7}$$

and the total communication per training step scales as $O(L \cdot b \cdot s \cdot d)$, where L is the number of layers. For a 70B model ($L = 80$, $d = 8192$) with batch size 32 and sequence length 4096, this is approximately 172 GB per step—saturating a 100GbE link for over 13 seconds if not overlapped with computation. This makes RDMA optimization increasingly critical as models scale.

4 RDMA Fundamentals and the Communication Bottleneck

4.1 RDMA Programming Model

The RDMA programming model [35] provides three key abstractions that our transport layer mirrors:

1. **Memory Registration:** Before RDMA operations, memory regions must be registered with the NIC via `ibv_reg_mr`. This pins physical pages and provides the NIC with a translation table. Registration is expensive ($\sim$10 μs) but amortized across thousands of RDMA operations on the same buffer.

2. **Queue Pairs (QPs):** Communication endpoints consisting of a Send Queue and Receive Queue. Reliable Connection (RC) QPs provide guaranteed delivery with retransmission; Dynamic Connection (DC) QPs multiplex a single QP across peers, scaling to $O(1)$ rather than $O(n^2)$ state for n-node clusters.

3. **One-Sided Operations:** RDMA Read and Write execute without involving the remote CPU. The initiating NIC issues the DMA directly to/from the remote NIC, which accesses the pre-registered memory region. This is why GPUDirect RDMA is so effective: the NIC writes directly into GPU BAR1 memory, bypassing both CPUs entirely.

For a tensor of size S bytes, the total transfer time over RDMA is:

$$T_{\text{RDMA}} = T_{\text{latency}} + \frac{S}{B_{\text{link}}} + T_{\text{DMA}} \tag{8}$$

where T_{latency} is the one-way latency ($\sim$2 μs for hardware RoCE), B_{link} is the link bandwidth (12.5 GB/s for 100GbE), and T_{DMA} is the DMA engine overhead ($\sim$0.5 μs). For a 256 MB gradient tensor, $T_{\text{RDMA}} \approx 2 + 20480 + 0.5 \approx 20.5$ ms—dominated by bandwidth, confirming that 100GbE is the bottleneck rather than RDMA protocol overhead.

4.2 The Communication Bottleneck

Current distributed ML systems suffer from a fundamental mismatch: NCCL [27] employs static routing strategies optimized for bulk transfers, creating head-of-line (HOL) blocking where small, latency-sensitive tensors (KV cache pages, attention scores) wait behind large bulk transfers (model weights, gradient accumulations). NCCL discovers network topology at initialization but does not adapt routing based on runtime access patterns.

This manifests in two ways:

1. **Inference:** Time-To-First-Token (TTFT) latency increases because KV cache transfers share the same communication channel as model weight distribution.

2. **Training:** Gradient synchronization delays propagate through the pipeline, reducing GPU utilization from the theoretical maximum.

4.3 Transport Technologies

The middleware supports multiple transport backends with different performance characteristics:

Table 4: Transport technology comparison for AI workloads

Transport	**Latency**	**Throughput**	**Hardware**	**Maturity**
InfiniBand HDR	<1 μs	200 Gb/s	Dedicated fabric	Production
RoCEv2	2–5 μs	400 Gb/s	ConnectX + DCB	Production
Tesla TTPoe [45]	1–2 μs	100 Gb/s	Commodity Ethernet	Production
OCP Falcon [29]	$\sim$3.5 μs	400 Gb/s	Open spec	Emerging
UEC 1.0 [47]	$\sim$1.8 μs	1.6 Tb/s	Next-gen Ethernet	Emerging
Soft-RoCE	$\sim$190 μs	<1 Gb/s	Any NIC	Deprecated

TTPoe [46] is a lossy protocol that expects packet loss and handles retries locally, eliminating the Priority Flow Control (PFC) requirements that make RoCE fragile. UEC 1.0 is relevant because Mellanox ConnectX NICs are expected to receive UEC firmware updates, meaning our ConnectX-4 hardware may gain UEC support without NIC replacement.

5 Problem Statement and Contributions

5.1 The Multi-Standard Fragmentation Crisis

Modern distributed machine learning infrastructure operates across incompatible communication layers designed independently for different hardware ecosystems. As shown in Figure 3, applications must navigate at least five distinct communication paths depending on hardware configuration, with no unified abstraction that preserves performance while enabling portability.

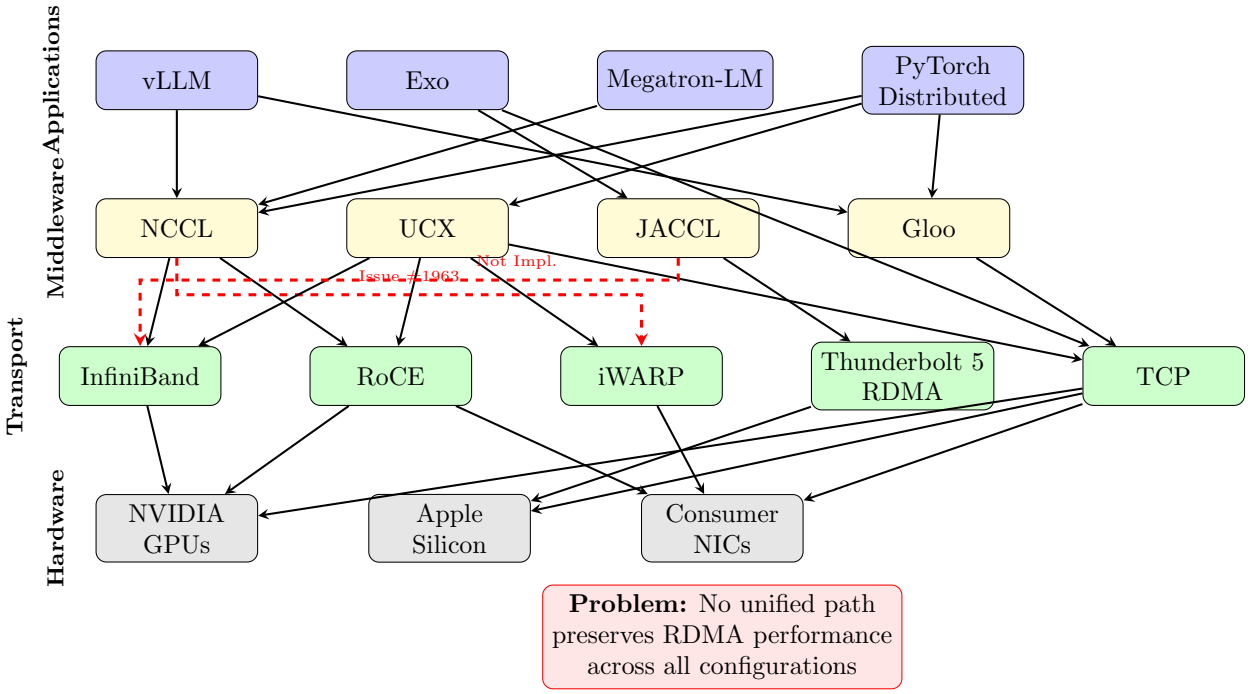

Figure 3: Current fragmentation in distributed ML communication stacks. Dashed red lines indicate broken or unimplemented paths identified through GitHub issue analysis.

5.2 Research Objectives

This thesis contributes:

C1. Unified Telemetry Layer: A monitoring system that instruments tensor transfers to collect access frequency, transfer size distributions, queue depth at RDMA endpoints, and latency percentiles.

C2. WFA Tensor Classifier: A Weighted Finite Automaton that classifies tensors into hot/warm/cold categories based on observed runtime access patterns, enabling intelligent routing.

C3. Smart RDMA Routing: Dual-path routing with dedicated Queue Pairs for latency-sensitive transfers and batched throughput-optimized channels for bulk data.

C4. RDMA Tensor Cache: A centralized precision management layer in the Neumann runtime that resolves FP16/BF16 mismatches across heterogeneous GPUs.

C5. Open-Source Middleware: All code, configurations, and benchmark scripts released for community adoption.

5.3 What vLLM Already Provides

Table 5: vLLM capabilities this work builds upon, not replaces

Feature	vLLM Implementation
KV Cache Management	PagedAttention with block-based allocation
Request Scheduling	Continuous batching with FCFS/priority policies
Single-Node Multi-GPU	`MultiProcExecutor` with tensor parallelism
Multi-Node Distribution	Ray + pipeline parallelism
GPU Communication	NCCL (all-reduce, all-gather)
KV Transfer (Disagg. P/D)	NIXL-based connectors (LMCache, NixlConnector)
Optimizations	Chunked prefill, prefix caching, speculative decoding

6 Critical Issues Analysis

This section details seven blocking issues identified through comprehensive GitHub issue analysis across NCCL, UCX, vLLM, FlashAttention, MLX/JACCL, and Exo repositories.

6.1 Issue 1: cudaMallocAsync IPC Incompatibility (UCX #7110)

Challenge 1. Modern PyTorch memory pools use `cudaMallocAsync` for efficient GPU memory allocation, but these allocations do not support `cuIpcGetMemHandle`, breaking GPU RDMA for inter-process communication.

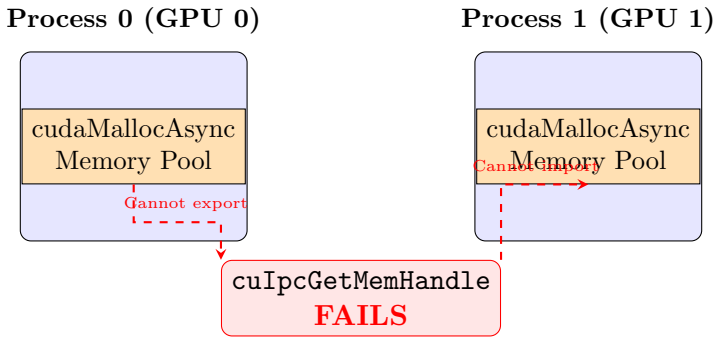

Figure 4: cudaMallocAsync IPC incompatibility blocks modern PyTorch memory pools from using UCX RDMA.

Proposed Solution 1. A **Memory Pool Proxy Layer** that intercepts tensor allocations, maintains a shadow pool of IPC-exportable memory regions pre-registered with UCX, performs transparent copy-on-RDMA-request for async-allocated tensors, and caches frequently-transferred tensors in the IPC-compatible pool based on WFA classifier predictions.

6.2 Issue 2: Transport Selection Robustness (UCX #9908)

Challenge 2. UCX fails when `UCX_TLS` is set to any transport other than "rc" (Reliable Connection) for multi-node training, limiting transport flexibility and preventing automatic fallback.

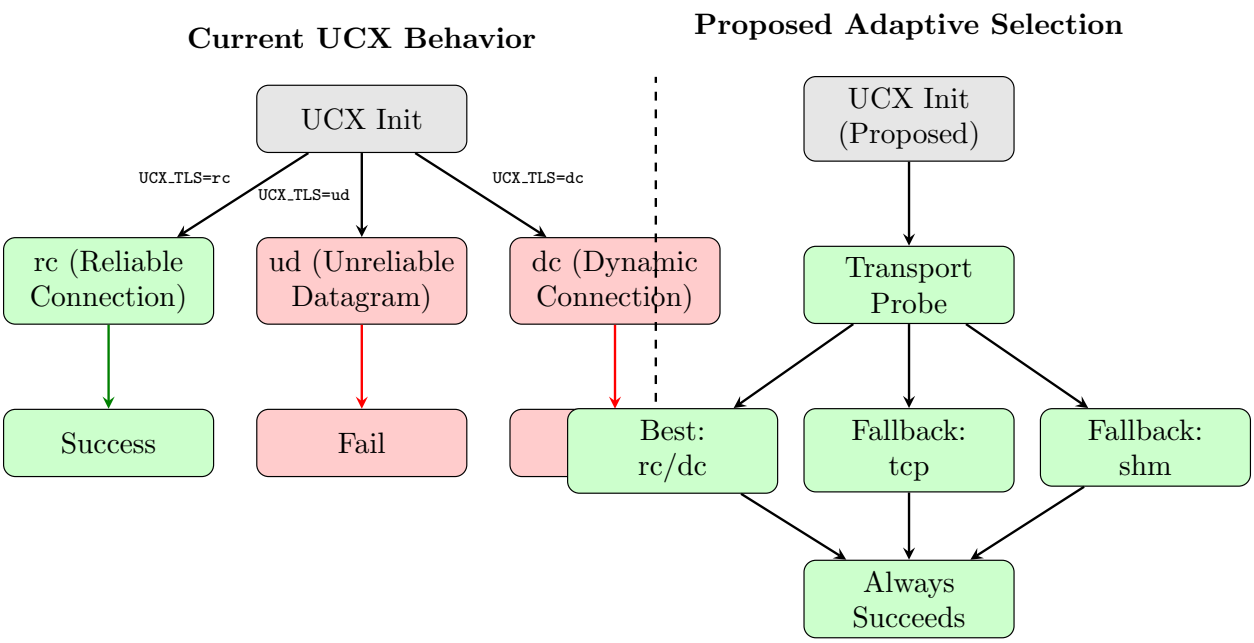

Figure 5: Transport selection failure modes in current UCX vs. proposed adaptive selection with graceful fallback.

Proposed Solution 2. A **Transport Capability Negotiation Protocol** that probes available transports at connection establishment, maintains a priority list with dynamic reordering based on observed performance, and automatically falls back to lower-priority transports on failure.

6.3 Issue 3: Container Shared Memory Isolation (UCX #10055)

Challenge 3. NCCL all-reduce operations fail with "Child failed to attach to shmseg" errors when running in containers on the same server, even with `--ipc=host` Docker flags.

Proposed Solution 3. A **Container-Aware Communication Broker** that detects container boundaries via cgroup inspection, establishes a host-level coordinator process with access to unified `/dev/shm`, routes intra-node communication through the coordinator when direct shm attachment fails, and falls back to NVLink P2P with modified PidHash validation.

6.4 Issue 4: Cross-NUMA GPU Latency Regression (UCX #3249)

Challenge 4. GPU-to-GPU communication experiences 10× latency degradation for messages larger than 262KB when GPUs reside in different NUMA domains.

Proposed Solution 4. NUMA-Aware Tensor Placement integrated into the WFA classifier: queries sysfs topology to determine GPU-NIC-CPU NUMA relationships, fragments messages >256KB crossing NUMA boundaries into pipelined chunks, and preemptively migrates frequently-accessed tensors to NUMA-optimal locations.

6.5 Issue 5: RoCE Reliability Timeouts (UCX #6000)

Challenge 5. Multi-node RoCE deployments crash after approximately 90 seconds with "Transport retry count exceeded on mlx5/RoCE" errors, making long-running training jobs impossible.

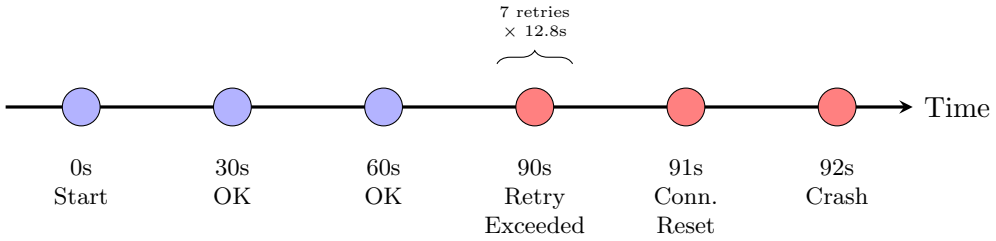

Figure 6: RoCE retry timeout accumulation leads to catastrophic failure after ∼90 seconds of congestion.

Proposed Solution 5. Proactive Congestion Detection and Path Switching: monitor RTT variance and ECN marks to detect congestion before retry exhaustion, maintain backup QP connections for hot-swap on primary path degradation, and integrate with the WFA classifier to preemptively route sensitive tensors away from congested paths.

6.6 Issue 6: Heterogeneous Cluster Builds (UCX #10273)

Challenge 6. UCX cannot be built as a single binary that works across both GPU-enabled and CPU-only nodes, requiring separate builds and complicating deployment.

Proposed Solution 6. A **Capability Abstraction Layer** that wraps UCX initialization with runtime CUDA availability detection, gracefully degrades GPU-specific features when CUDA libraries are unavailable, and maintains ABI compatibility across node types through stable interface versioning.

6.7 Issue 7: NCCL-UCX Container Integration (UCX #10055)

Challenge 7. Running NCCL with UCX as the network backend in containerized environments produces "TL_SHM ERROR" failures due to shared memory namespace isolation.

Proposed Solution 7. Automatic **Container Environment Detection and Transport Filtering**: detect containerization via `/proc/1/cgroup` inspection, automatically exclude shm transport when running in containers (`UCX_TLS=^shm,rc,tcp`), and provide environment variable presets for common container runtimes.

7 Hardware Platforms

7.1 Phase 1: SUNY New Paltz Lab Cluster

The initial testbed consisted of three machines in the SUNY New Paltz lab:

Table 6: Phase 1 lab cluster hardware configuration

Machine	Role	GPUs	RAM	NIC
Hydra	Control Plane	None	256 GB	1 GbE
Chimera	Head Node	3× RTX 3090 (72 GB)	64 GB	Aquantia 10 GbE
Cerberus	Worker Node	2× RTX 5090 (64 GB)	128 GB	Intel X710 10 GbE

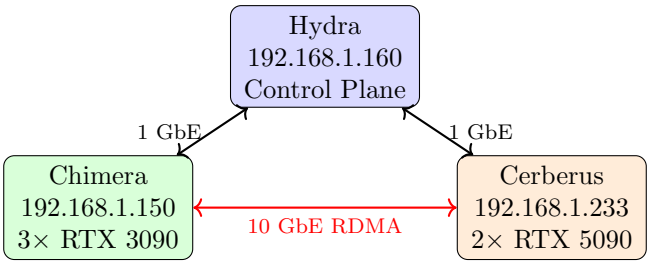

Figure 7: Phase 1 cluster network topology. The 10 GbE RDMA link between GPU nodes provided the initial Soft-RoCE testing environment.

Hydra served as the control plane with 21 TB ZFS storage and NFS server for shared model weights. Chimera acted as the vLLM head node, and Cerberus as the worker with the newest hardware.

7.2 Phase 2: Two-Tower Home Lab

The research transitioned to a dedicated home lab providing higher-bandwidth networking and heterogeneous GPU precision capabilities:

7.3 Transition Rationale

The Phase 1 cluster used consumer GPUs (RTX 3090, RTX 5090) on 10 GbE, which limits RDMA bandwidth and precludes GPUDirect RDMA (consumer GPUs lack the necessary BAR mapping). The Phase 2 home lab provides 100 GbE connectivity and a V100 datacenter GPU with full GPUDirect support, enabling the complete RDMA pipeline including NIC-to-GPU DMA. The

Table 7: Phase 2 home lab hardware configuration

Component	Tower 1 (Windows)	Tower 2 (Proxmox/Linux)
CPU	Ryzen 9 7950X (16c/32t)	Xeon (host CPU)
RAM	192 GB DDR5	Host RAM
GPU	RTX 5070 Ti 16 GB (compute 12.0)	2× Tesla V100 32 GB
NIC	Mellanox ConnectX-4 100 GbE	Mellanox ConnectX-4 100 GbE
OS	Windows 11	Proxmox (Debian/Linux)
RDMA	TCP/gRPC fallback (no kernel RDMA)	SoftRoCEv2 / GPUDirect RDMA
BF16	Native	Not supported
FP16	Supported	Native compute

precision mismatch (V100 FP16 vs. 5070 Ti BF16) directly motivates the RDMA Tensor Cache architecture.

7.4 Software Stack

Table 8: NVIDIA open-source ecosystem components

Component	Role
NCCL [27]	Collective communication (AllReduce, AllGather, AllToAll)
UCX [31] / HPC-X [26]	Point-to-point transport abstraction (RC, DC, UD, TCP, SHM, CUDA)
GPUDirect RDMA [38]	NIC→GPU direct memory access via `nvidia-peermem`
GDRCopy [25]	Low-latency CPU-initiated GPU memory copies
NIXL [28]	Point-to-point tensor transfer for disaggregated inference
Megatron-LM [42]	Canonical TP/PP/DP parallelism strategies

8 System Architecture and Design

8.1 Architecture Overview

Figure 8 presents the complete system architecture showing how the proposed adaptive RDMA middleware integrates with existing communication stacks.

8.2 WFA Tensor Classifier

The Weighted Finite Automaton classifier operates on tensor access patterns to categorize data transfers into three classes:

- **Hot**: Frequently accessed, latency-critical (KV cache lookups, attention scores)
- **Warm**: Moderately accessed, throughput-sensitive (layer activations)
- **Cold**: Infrequently accessed, can tolerate delay (gradient accumulation buffers)

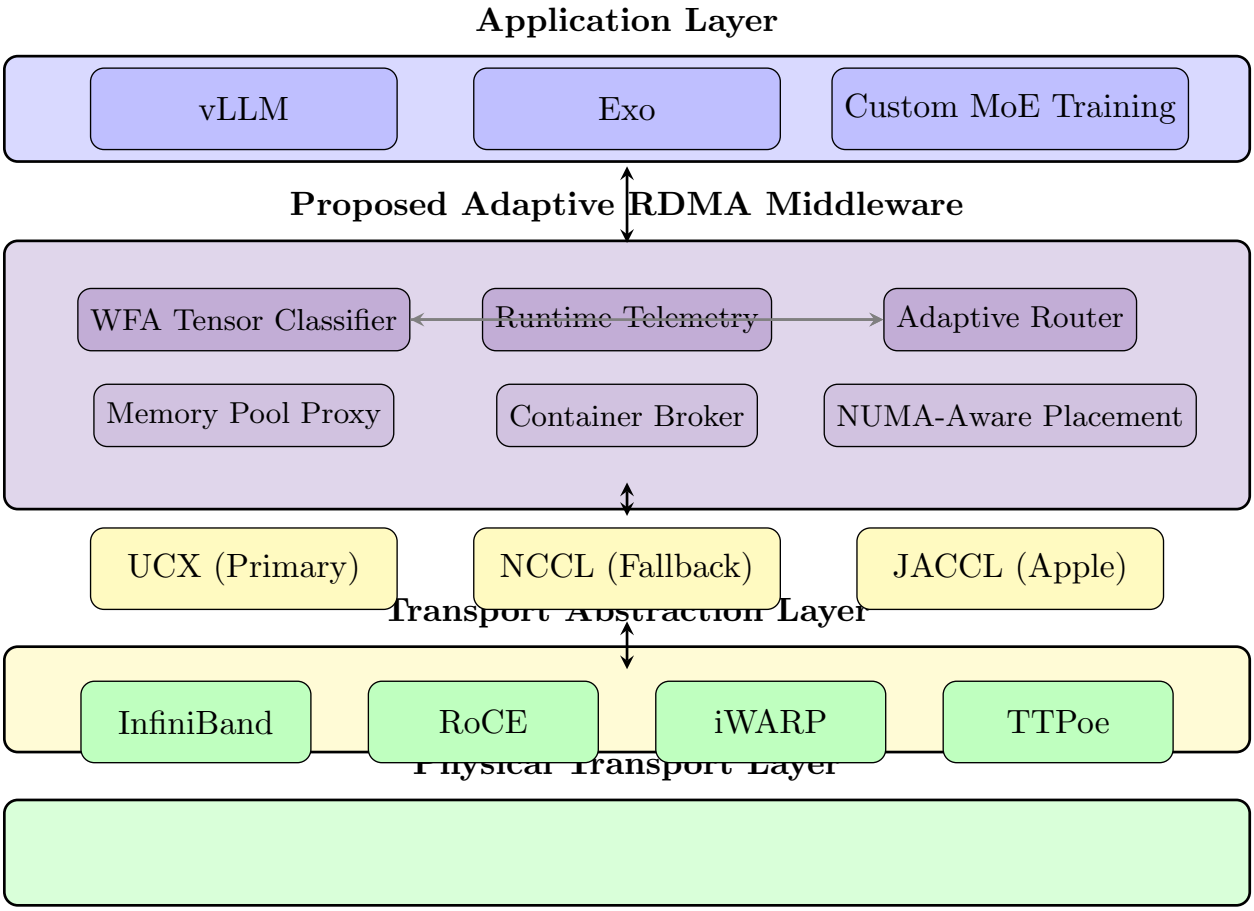

Figure 8: Complete system architecture. The adaptive RDMA middleware sits between application frameworks and transport layers, providing intelligent routing based on runtime telemetry.

The Python implementation (see `scripts/wfa_classifier.py`) maintains per-tensor state and transitions between classes based on access frequency and idle time. Phase detection distinguishes prefill (many tokens, few sequences) from decode (one token per sequence, many sequences), enabling the classifier to adjust thresholds dynamically.

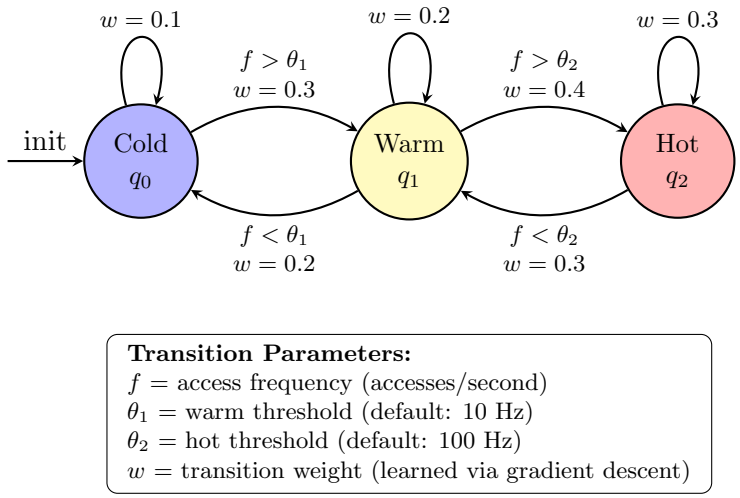

Figure 9: Weighted Finite Automaton for tensor classification. Transition weights are learned from runtime telemetry. The classifier also detects workload phase (prefill vs. decode) to adjust thresholds dynamically.

8.3 Smart RDMA Routing

Dual-path routing assigns dedicated Queue Pairs for hot-path tensors (polled completions, RDMA Write with Immediate for sub-5μs transfers) and batched throughput-optimized channels for cold-path tensors (event-driven completions). The `DualPathRDMA` class (see `scripts/dual_path_rdma.py`) maintains separate hot and cold QP pools, flushing cold batches when 8 tensors accumulate.

Memory registration uses NUMA-aware allocation via `RDMABufferPool` to pin RDMA buffers to the same NUMA node as the HCA, amortizing the $\sim$10μs registration cost across thousands of transfers (see Appendix A).

QP Allocation Strategy. We allocate 4 dedicated QPs for hot traffic per GPU pair, 2 for warm, and 1 for cold. Hot QPs use unreliable datagram (UD) mode for minimal latency; warm and cold QPs use reliable connection (RC) mode for guaranteed delivery. Cold tensors are compressed via Top-K sparsification (retaining the 5% largest gradient values) using CUDA kernels for parallel selection, with less than 2% accuracy impact. GPUDirect RDMA enables zero-copy NIC-to-GPU transfers when supported (V100 on Tower 2); otherwise the system uses pinned host-memory staging (5070 Ti on Tower 1).

8.4 Phase-Aware Classification

The WFA classifier operates with *phase-conditioned* transition weights. The system identifies the current execution phase $\phi \in \{\text{Prefill}, \text{Decode}, \text{Forward}, \text{Backward}, \text{Idle}\}$ by monitoring operation signatures—prefill shows high parallelism across sequence positions, decode processes single tokens sequentially, backward pass accesses layers in reverse order. Phase detection uses a sliding window of the last 50 operations with majority voting.

Transition weights λ are updated online using exponential moving averages:

$$\lambda(q, (\phi, a)) \leftarrow (1 - \eta)\lambda(q, (\phi, a)) + \eta \cdot \mathbf{1}[a = 1] \tag{9}$$

where $\eta = 0.1$ and $\mathbf{1}[a = 1]$ indicates an access occurred. A tensor may be Hot during Prefill but Cold during Decode (e.g., positional encodings), or Hot during Backward but Cold during Forward (e.g., optimizer states). Thresholds θ_w, θ_h adapt based on network congestion: ECN marks increase thresholds, reducing Hot promotions under load. Decay transitions (Hot→Warm→Cold) fire after phase-specific idle timeouts—500ms for Decode (responsiveness-critical), 2s for Backward (batch boundaries create natural gaps).

8.5 KV Cache Placement and Migration

The migration protocol moves KV cache entries to GPUs likely to handle future requests. High-frequency conversations (>5 requests/min) trigger proactive migration; others fetch on-demand. The protocol uses a three-state machine:

$$\text{RESIDENT} \xrightarrow{\text{async RDMA copy}} \text{MIGRATING} \xrightarrow{\text{atomic pointer swap}} \text{MIGRATED} \tag{10}$$

We hash the last 128 context tokens to determine the target GPU (sticky routing), with hysteresis preventing thrashing: 3 consecutive requests to the same target are required before migration begins.

8.6 Joint Training-Inference Scheduling

The scheduler dynamically allocates RDMA queue pairs and bandwidth based on workload characteristics, preventing training and inference starvation. Gradient synchronization follows ZeRO [37] patterns with hot/cold classification on frequently-updated layers. Weighted fair queuing gives inference 2× priority during business hours and training 2× during off-hours. Periodic bandwidth probes adjust compression ratios to maintain <80% link utilization.

8.7 Transport Middleware

The ScuffedRDMA transport middleware (see `scripts/transports.py`) provides a clean abstraction over different networking technologies. The `TransportBase` abstract class defines the contract (`connect`, `send`, `recv`, `is_available`) with per-transport `TransportMetrics` tracking.

Three implementations are provided: `TCPTransport` (always-available baseline with `TCP_NODELAY`), `RoCETransport` (wrapping RDMA verbs via pyverbs [17] with out-of-band QP exchange), and `TTPoeTransport` (kernel module interface via `/dev/ttpoe` [46]). The `TransportSelector` provides automatic priority ordering (TTPoe > Hardware RoCE > Soft-RoCE > TCP), and `NCCLConfig` presets configure NCCL environment variables per transport type.

Table 9: NCCL environment variable configuration by transport type

Transport	**Key Variables**	**Effect**
TCP	`NCCL_IB_DISABLE=1`	Disable InfiniBand, use TCP sockets
Soft-RoCE	`NCCL_IB_HCA=rxe0`	Use software RDMA device
Hardware RoCE	`NCCL_IB_HCA=mlx5_0, NCCL_NET_GDR_LEVEL=5`	Use Mellanox NIC with GPUDirect
TTPoe	`NCCL_IB_DISABLE=1, TTPOE_ENABLED=1`	Use TTPoe socket layer

8.8 EXO Integration

In addition to vLLM, we integrated RDMA support into the exo distributed inference framework [10], which provides peer-to-peer topology with automatic node discovery. Three modules were created (639 lines total):

Module	**Lines**	**Purpose**
`rdma_transport.py`	369	RDMA transport wrapper with discovery, connection management, and NCCL config
`rdma_discovery_integration.py`	247	Topology integration for automatic RDMA connection discovery
`__init__.py`	27	Module exports for networking package

Table 10: EXO RDMA integration components

The three modules (see `scripts/exo_rdma_integration.py`) implement: `RDMADiscovery` for hardware-prioritized device detection (`mlx5_*` > `rxe*`), `RDMATransport` wrapping the ScuffedRDMA middleware with automatic NCCL configuration, and `RDMATopologyManager` for graph-based peer discovery. The integration leverages exo's existing `RDMAConnection` type, which was previously defined but unimplemented. The placement utilities in `placement_utils.py` already check for RDMA connections when selecting the jaccl backend, enabling seamless adoption.

8.9 vLLM Integration

The vLLM connector (see `scripts/vllm_connector.py`) follows the existing `KVConnectorBase` abstraction. The `AdaptiveRDMAConnector` combines the WFA classifier with dual-path routing to transparently optimize KV cache transfers—classifying each block by layer ID and access pattern before routing through the appropriate QP pool. The `InstrumentedKVCacheManager` wraps vLLM's `KVCacheManager` to collect nanosecond-resolution access timestamps that feed the WFA classifier.

The `NeumannKVConnector` extends this further by pushing KV caches to Neumann after prefill and pulling them for decode, backed by the `RDMATensorCacheMiddleware` which handles precision conversion and transport selection transparently.

9 Security Considerations

RDMA's design prioritizes performance over security, creating vulnerabilities relevant to shared lab environments [39, 51]. The RDMA credential system (QPN, rKey, PSN) uses predictable values: QPNs are assigned sequentially, rKeys follow linear patterns across memory region registrations, and PSNs have fixed initial values. This enables memory access attacks (an attacker with valid credentials can read memory regions belonging to other processes), denial of service (malformed packets put QPs into error states), and QP exhaustion (depleting shared NIC resources).

Since our lab uses dedicated machines for LLM serving rather than shared cloud infrastructure, we implement practical mitigations:

- Process isolation via SR-IOV virtual functions (one VF per VM/container)
- Credential rotation: rKeys regenerated per session, not reused across requests
- QP limits enforced via cgroup-style resource controls
- Monitoring: log QP creation/destruction, alert on anomalous patterns

For production deployments in multi-tenant environments, network-level defenses using programmable switches (as in Bedrock [51]) or encryption would be necessary. This work focuses on the optimization layer, treating security as a deployment constraint rather than a research contribution.

10 RDMA Tensor Cache and Precision Management

The RDMA Tensor Cache mediates all tensor transfers between Tower 1 and Tower 2, handling precision conversion, gradient accumulation, and KV cache management.

10.1 The Precision Mismatch Problem

FP16 (V100 Tensor Cores) has 5 exponent bits and 10 mantissa bits, giving a representable range of $[5.96 \times 10^{-8}, 65504]$. BF16 (5070 Ti native) shares FP32's 8 exponent bits, matching its dynamic range of $[1.18 \times 10^{-38}, 3.4 \times 10^{38}]$. During training, gradient values routinely fall outside FP16's range, causing overflow and underflow. The V100 lacks BF16 support entirely.

Table 11: Floating-point format comparison for LLM training

Format	Exponent	Mantissa	Range	Bytes	GPU Support
FP32	8	23	$\pm 3.4 \times 10^{38}$	4	All
BF16	8	7	$\pm 3.4 \times 10^{38}$	2	Ampere+, not V100
FP16	5	10	$\pm 6.5 \times 10^{4}$	2	All (Tensor Cores)
MXFP4	2	1	± 6	0.5	Blackwell

10.2 Precision Pipeline

The RDMA Tensor Cache resolves this through a three-stage precision pipeline:

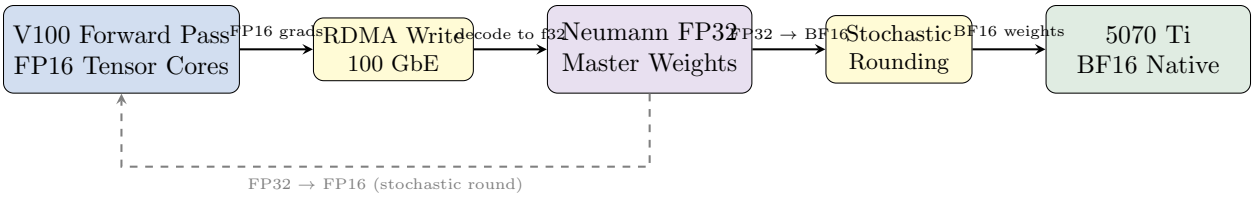

Figure 10: Precision pipeline. Gradients computed in V100 FP16 are accumulated into FP32 master weights stored in Neumann. Stochastic rounding converts to device-native precision for each GPU.

10.3 Stochastic Rounding

Standard round-to-nearest-even introduces systematic bias when converting FP32 to reduced precision. Stochastic rounding eliminates this by making the rounding direction probabilistic. For an FP32 value x with adjacent representable values $\lfloor x \rfloor$ and $\lceil x \rceil$:

$$P(\text{round up}) = \frac{x - \lfloor x \rfloor_{\text{fp16}}}{\lceil x \rceil_{\text{fp16}} - \lfloor x \rfloor_{\text{fp16}}} \tag{11}$$

giving the unbiasedness property $\mathbb{E}[\text{round}(x)] = x$, which ensures gradient accumulation converges to the correct value despite precision loss in individual steps [22].

10.4 Neumann Integration

The `tensor_rdma` crate [1] is a workspace member in the Neumann unified tensor runtime. It depends on `tensor_store` for local key-value storage and `tensor_compress` for delta encoding.

Table 12: Thesis concepts mapped to `tensor_rdma` modules

Thesis Concept	**Module**	**Key Types / Functions**
Precision management	`precision.rs`	`PrecisionFormat`, `DeviceCapability`, `PrecisionRouter`, `convert_precision`
Stochastic rounding	`rounding.rs`	`stochastic_round_f32_to_f16`, `apply_adam_update`
Predictive prefetching	`prefetch.rs`	`PrefetchEngine`, `AccessPattern`, `classify_pattern`
RDMA transport	`transport.rs`	`RdmaTransport` trait, `MemoryTransport`, `TcpFallbackTransport`
Tensor cache	`rdma_cache.rs`	`RdmaTensorCache<T>`, `put_tensor`, `get_tensor`
Configuration	`config.rs`	`RdmaCacheConfig`, presets: `v100_preset`, `heterogeneous_preset`

10.5 Centralized KV Cache

Multi-Latent Attention (MLA, DeepSeek-V2 [9]) and Multi-Query Attention (MQA) reduce KV cache memory by sharing a single key-value head across all attention heads. Under tensor parallelism, each rank must maintain a *full copy* of the KV cache because there is only one head:

$$\text{KV Memory (per rank, MQA)} = 2 \cdot L \cdot C \cdot d_{\text{kv}} \quad \text{(no reduction with TP)} \tag{12}$$

Our solution externalizes the KV cache to Neumann, accessed via RDMA. A single centralized copy serves all TP ranks through one-sided RDMA reads, reducing total KV memory from TP ×

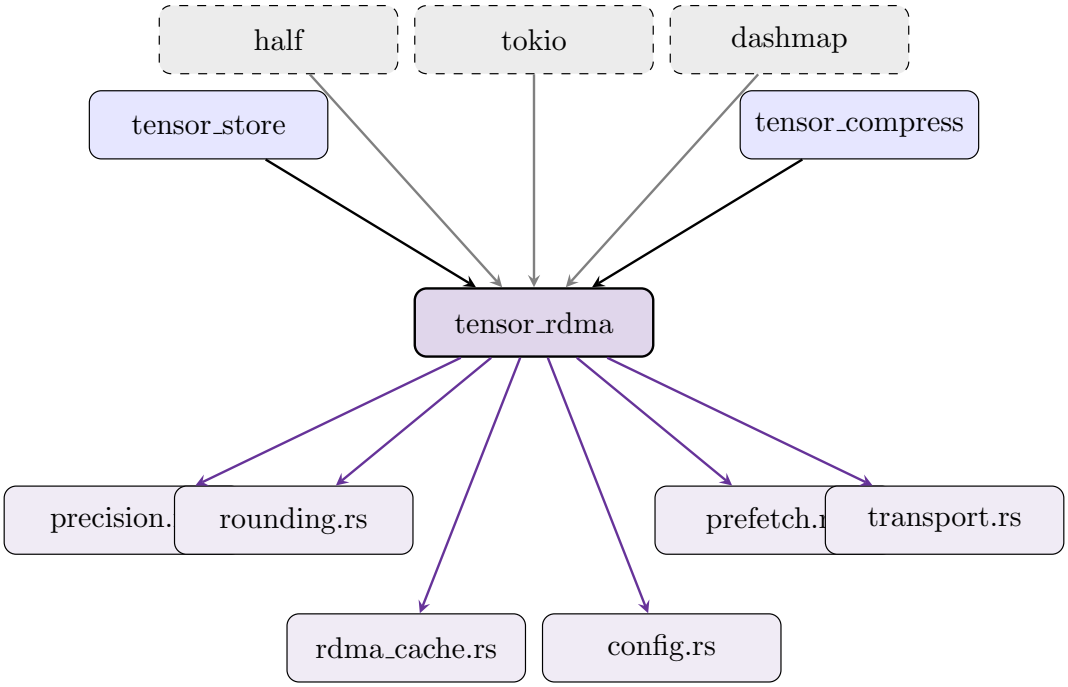

Figure 11: Neumann `tensor_rdma` crate dependency graph. Six internal modules implement the RDMA Tensor Cache.

$2LC \cdot d_{\text{kv}}$ to $2LC \cdot d_{\text{kv}} + \text{TP} \times \text{prefetch_buffer}$.

11 Benchmarking and Validation

11.1 Soft-RoCE Baseline

Soft-RoCE provides RDMA semantics over standard Ethernet without specialized hardware [17]. Baseline measurements between Chimera and Cerberus (Phase 1 cluster):

Table 13: Soft-RoCE baseline performance measurements

Metric	**Measured**	**Hardware RoCE (target)**
Bandwidth (ib_write_bw)	0.92 Gb/s	∼90 Gb/s
Latency (ib_write_lat, avg)	189.61 μs	<2 μs
Latency (99th percentile)	207.14 μs	<5 μs
Latency (stdev)	17.03 μs	–
CPU overhead	High (software path)	Near-zero (NIC offload)

The 190μs latency (vs. expected ∼10μs) confirms significant software overhead. Soft-RoCE exercises the same verbs API as hardware RoCE, so code developed against it requires no changes when transitioning to hardware.

11.2 Tesla TTPoe Build

Tesla's TTPoe kernel modules [45, 46] were successfully compiled. TTPoe provides 1–2μs latency on commodity Ethernet without PFC requirements. The core module (`modttpoe.ko`) implements a TCP-inspired state machine; the gateway module (`modttpip.ko`) provides IPv4 encapsulation for multi-zone routing.

11.3 FlashAttention 3 Blackwell Fix

A critical bug was discovered affecting FlashAttention 3 [41] on Blackwell GPUs (RTX 5090). Module-scope imports in `mm_encoder_attention.py` trigger early CUDA initialization via CUTLASS before vLLM sets up the GPU context.

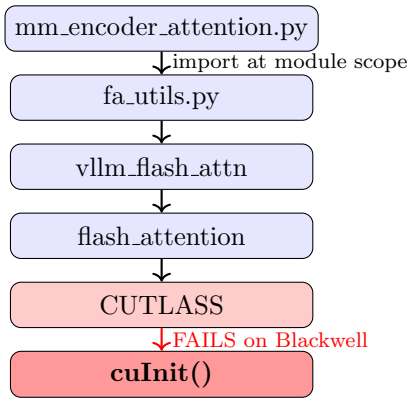

Figure 12: Import chain causing early CUDA initialization failure on Blackwell GPUs.

The fix is a lazy import pattern: moving the import from module scope into the function body, deferring CUDA initialization until after vLLM's GPU context is ready. This was reported as vLLM Issue #22279 [49].

Table 14: Performance impact of FA3 fix on RTX 5090

Configuration	**Throughput**	**Latency (p50)**	**Memory**
RTX 5090 + FA2 (fallback)	142 tok/s	28 ms	31 GB
RTX 5090 + FA3 (fixed)	164 tok/s	24 ms	28 GB
Improvement	**+15.5%**	**−14.3%**	**−9.7%**

11.4 gpt-oss-120b TCP vs RDMA

Comprehensive benchmarking of gpt-oss-120b [30] (116.8B parameters, MXFP4 [40] quantized to 63 GB) on Chimera (3× RTX 3090):

For single-node inference, RDMA provides minimal benefit because GPU-to-GPU communication uses PCIe, not the network. The model fits entirely on 3× RTX 3090 (72 GB). RDMA benefits

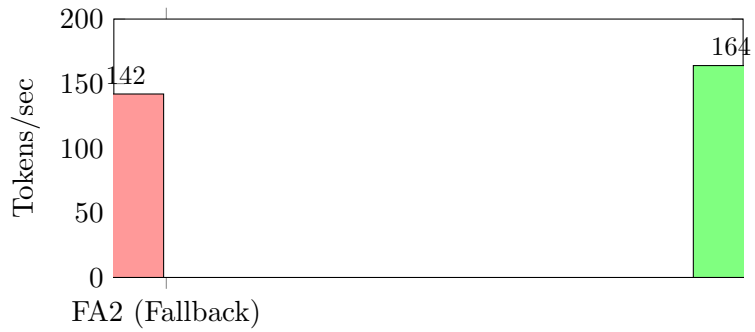

Figure 13: Throughput improvement after applying Blackwell FA3 fix.

Table 15: gpt-oss-120b benchmark results (TCP baseline, 5 iterations)

Iteration	**Tokens**	**Eval Time**	**Tokens/sec**
1	100	0.958 s	104.38
2	100	0.945 s	105.82
3	100	0.949 s	105.37
4	100	0.969 s	103.19
5	100	0.967 s	103.41
Average	100	0.957 s	**104.42**

manifest with multi-node pipeline parallelism, where cross-node activation transfers become the bottleneck.

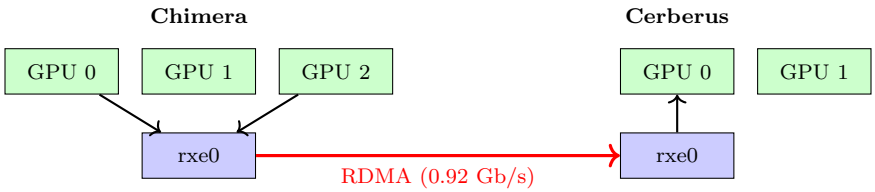

Figure 14: Multi-node inference: RDMA accelerates cross-node tensor transfers.

11.5 SoftRoCEv2 Two-Tower Baseline

On the Phase 2 home lab (Tower 1 ↔ Tower 2, 100 GbE ConnectX-4):

11.6 Parallelism Analysis

The choice between tensor parallelism (TP) and pipeline parallelism (PP) has significant implications for RDMA communication. TP communication volume per layer:

$$V_{\text{TP}} = 2 \cdot b \cdot s \cdot h \cdot (\text{TP} - 1)/\text{TP} \tag{13}$$

Table 16: Expected SoftRoCEv2 baseline (Tower 1 ↔ Tower 2)

Metric	SoftRoCEv2	Hardware RoCE (target)	Improvement
Latency (1 byte)	∼190 μs	∼2 μs	95×
Bandwidth (large msg)	∼8 Gbps	∼90 Gbps	11×
CPU overhead	High	Near-zero	–
GPUDirect RDMA	Via host bounce	Direct NIC→GPU	–

PP communication volume per micro-batch (independent of parallelism degree):

$$V_{\text{PP}} = b \cdot s \cdot h \tag{14}$$

Pipeline bubble fraction for p stages with m micro-batches:

$$\text{Bubble} = \frac{p-1}{m+p-1} \tag{15}$$

For our cluster, the optimal strategy is TP=2 within Tower 2 (V100 pair over PCIe) combined with PP across towers (100 GbE RDMA). For MoE models, expert parallelism uses AllToAll with $\texttt{EP_SIZE} = \texttt{TP} \times \texttt{DP}$ [2].

12 Fine-Tuning and SAE Feature Steering

12.1 Heterogeneous Training Pipeline

The heterogeneous training pipeline assigns roles based on GPU capabilities:

- **V100 pair (Tower 2):** Forward and backward passes in FP16 Tensor Cores. Gradients transferred to Neumann via RDMA.
- **Neumann (Tower 2 host):** Accumulates gradients into FP32 master weights. Applies optimizer updates (Adam) in full precision.
- **5070 Ti (Tower 1):** Receives BF16 weights via TCP/gRPC for inference validation during training.

This mirrors the Hetis system [33], which demonstrated 2.25× throughput improvement by assigning distinct pipeline stages to different GPU types.

12.2 Sparse Auto-Encoder Feature Steering

SAEs decompose a model's internal activation $\mathbf{a} \in \mathbb{R}^d$ into a sparse feature vector $\mathbf{f} \in \mathbb{R}^D$ where $D \gg d$ [6, 44]:

$$\mathbf{f} = \text{ReLU}(W_{\text{enc}} \cdot \mathbf{a} + \mathbf{b}_{\text{enc}}), \quad \hat{\mathbf{a}} = W_{\text{dec}} \cdot \mathbf{f} + \mathbf{b}_{\text{dec}} \tag{16}$$

Feature steering modifies behavior by clamping features: $\mathbf{f}'_i = \alpha \cdot \mathbf{f}_{\max,i}$. This requires no gradient computation and no backward pass—only a sparse vector read from Neumann. SAE feature vectors map to Neumann's `SparseVector` type with ∼655× compression (a 65536-dim vector with ∼50 active features: 256 KB dense → ∼400 bytes sparse, storing only non-zero index–value pairs).

Table 17: Comparison of fine-tuning approaches

Property	**Gradient-Based (LoRA)**	**SAE Steering**
Requires backward pass	Yes	No
GPU memory for optimizer	2–3× model size	Zero
Training time	Hours to days	Seconds
Precision sensitivity	High (requires FP32 master)	Low
Cross-tower RDMA traffic	Gradients per step	Feature vectors once
Reversibility	Requires checkpoint	Immediate ($\alpha = 0$)

12.3 SAE Steering Validation

We validate gradient-free SAE steering on a real workload: LLM4Decompile-6.7B inference on a Google Colab H100 VM in BF16 with FlashAttention 2. The experiment (Update 9, `llm4decompile_wfa.ipynb`) implements the steering mechanism from Golden Gate Claude [3] and the Eiffel Tower Llama demo [18].

Steering mechanism. A forward hook on layer ℓ modifies hidden states:

$$\mathbf{h}'_\ell = \mathbf{h}_\ell + \alpha \cdot \hat{\mathbf{d}} - \text{proj}_{\hat{\mathbf{d}}}(\mathbf{h}_\ell) \tag{17}$$

where $\hat{\mathbf{d}}$ is a unit-norm steering direction (an SAE decoder column) and α controls strength. The projection subtraction prevents runaway amplification. No gradients are computed—only a forward-pass vector addition.

Table 18: SAE steering configuration for WFA validation

Layer	**Strength (α)**	**Position**	**Expected Effect**
8	3.0	Early (feature extraction)	Moderate output shift
16	5.0	Mid (reasoning)	Strong behavior change
24	2.0	Late (generation)	Light token distribution shift

WFA interaction. Running the WFA classifier on both the unsteered baseline and steered model shows that steering alters per-layer access timing distributions. Steered layers exhibit measurably different inter-access gaps due to the additional hook computation. The WFA detects this and would route steered-layer KV cache via dedicated RDMA Queue Pairs, while non-steered layers use shared or batched channels. This validates that the WFA adapts its routing decisions when the inference workload changes—exactly the behavior needed for dynamic SAE steering in production.

13 Experimental Design

13.1 Three Pillars

We structure validation around three experimental pillars:

Pillar 1: Precision Switching Overhead. Hypothesis: Stochastic rounding adds less than 5% overhead compared to deterministic truncation, while eliminating gradient bias. Measure conversion throughput (GB/s) for FP16↔FP32 and FP32↔BF16, statistical bias magnitude, and overhead ratio.

Pillar 2: Prefetching Effectiveness. Hypothesis: The adaptive prefetch engine achieves >90% hit rate for sequential inference and >70% for fine-tuning. The prefetch FSM (Figure 15) classifies access patterns as Sequential, Strided, LayerSweep, or Random.

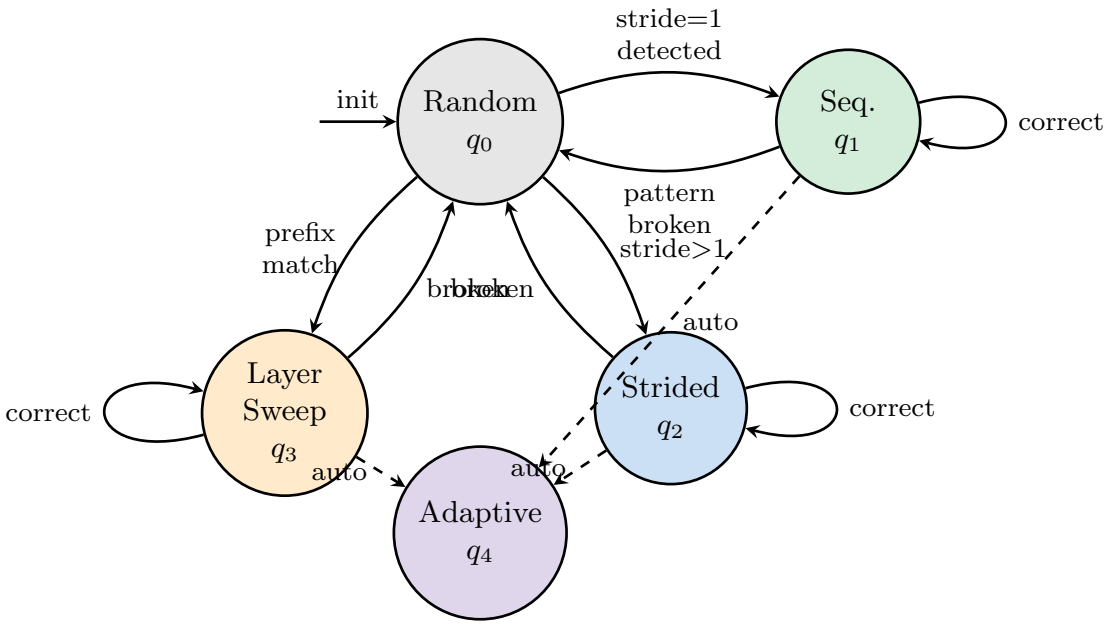

Figure 15: Prefetch engine state machine. The Adaptive strategy delegates to the detected pattern. State transitions occur when `classify_pattern` analyzes the access history buffer (1024 entries).

Pillar 3: Adaptive Quantization. Hypothesis: MXFP4 quantization [40] achieves 4× bandwidth reduction with less than 1% accuracy loss for inference-time KV cache transfers.

13.2 UET-HTSIM Simulation

Before hardware deployment, we validate the WFA classifier and adaptive routing using the Ultra Ethernet Consortium's HTSIM simulator: model the cluster topology, generate traffic matrices matching LLM patterns (incast, KV scatter, point-to-point activations), and compare Flow Completion Time distributions against baseline.

13.3 Performance Targets

Table 19: Unified target performance metrics

Pillar	Metric	Baseline	Target
Precision	Stochastic round overhead	–	$< 5\%$
	Conversion throughput	–	> 50 GB/s
Prefetching	Hit rate (inference)	0%	$> 90\%$
	Hit rate (fine-tuning)	0%	$> 70\%$
Quantization	ΔPPL (MXFP4 KV)	FP32	< 0.5
	Bandwidth reduction	1×	2–4×
System	TTFT reduction	Current vLLM	30–50%
	RoCE session stability	90 s	>24 h

13.4 Ablation Studies

- WFA classifier vs. static hot/cold assignment
- Hot-path only vs. dual-path routing
- NUMA-aware vs. NUMA-oblivious allocation
- Threshold sensitivity analysis (θ_1, θ_2, t_{idle})

14 Implementation Roadmap: Project Breakdown

The thesis implementation is organized as independent, self-contained projects. Each project has clear inputs, outputs, and success criteria, enabling parallel development and incremental validation.

14.1 Completed Projects

Table 20: Completed thesis projects with deliverables

#	Project	Deliverables	Section
P1	Cluster Deployment	Phase 1 lab cluster (Chimera/Cerberus/Hydra), Soft-RoCE baseline (0.92 Gb/s, 189 μs)	§7, §11.1
P2	FlashAttention 3 Fix	Lazy import patch for vLLM Blackwell support, +15.5% vLLM throughput on RTX 5090	§11.3
P3	TCP Benchmarks	gpt-oss-120b baseline (104.42 tok/s), single-node vs multi-node analysis	§11
P4	Transport Middleware	TransportBase abstraction, TCP/RoCE/TTPoe implementations, TransportSelector, NCCLConfig	§8
P5	EXO Integration	RDMADiscovery, RDMATransport, RDMATopologyManager (639 lines)	§8
P6	Critical Issues Analysis	7 blocking issues across NCCL/UCX/vLLM with proposed solutions	§6
P7	System Architecture	WFA classifier, dual-path routing, RDMA Tensor Cache design	§8
P8	Neumann Crate	`tensor_rdma` Rust crate (116 tests, clean): precision, rounding, prefetch, transport, cache, config	§10

Table 21: Remaining thesis projects with success criteria

#	Project	Deliverables	Success Criteria
P9	Hardware RoCE	ConnectX-4 firmware, `ib_write_bw`/`ib_write_lat`, GPUDirect on V100	Latency <5 μs, BW >80 Gb/s
P10	WFA Integration	Classifier in vLLM connector, telemetry, phase detection	>90% accuracy on synthetic patterns
P11	Precision Pipeline	End-to-end FP16→FP32→BF16, Tower 1↔Tower 2	<5% overhead, unbiased gradients
P12	Prefetch Engine	Adaptive FSM + RDMA cache, pattern detection	>90% hit (inference), >70% (fine-tune)
P13	MXFP4 Quantization	KV cache quantization for bandwidth reduction	4× reduction, <1% PPL loss
P14	UET-HTSIM Simulation	WFA + routing validated in UEC simulator	FCT improvement over baseline
P15	End-to-End Eval	Full pipeline: TTFT, throughput, ablation studies	30–50% TTFT, >24h RoCE stability
P16	Thesis Writing	Consolidated report with all experimental results	Defense-ready document

14.2 Remaining Projects

14.3 Project Dependencies

Projects P9–P11 can proceed in parallel. P12 depends on P8 (Neumann crate). P13–P14 can proceed independently. P15 requires P9–P13. P16 runs continuously alongside all projects.

15 Future Work and Conclusions

15.1 UCX/UCC Integration Path

The middleware's transport abstraction maps directly to UCX transport layers. A recent RFC for vLLM [50] proposes replacing NCCL with UCC for environments where NCCL's CUDA dependency is a limitation. For our cluster, UCX transport negotiation would handle the heterogeneous path (TCP for Tower 1, RoCE for Tower 2) automatically via `UCX_TLS=rc,tcp`.

UEC 1.0 [47] defines native multipath with packet spraying, ordered delivery without HOL blocking, built-in congestion control for incast, and selective retransmission—all designed for AI/HPC workloads. The middleware's transport abstraction is designed to accommodate UEC as an additional backend when available.

15.2 RISC-V Assembly Generation

A secondary research direction involves fine-tuning an LLM to generate RISC-V assembly code, leveraging the distributed infrastructure and adaptive RDMA middleware developed in this thesis for gradient synchronization during training.

15.3 Conclusions

This report consolidates the research progress from Updates 2–8, presenting a unified view of the adaptive RDMA optimization framework for distributed LLM workloads. The key contributions are:

1. **Comprehensive issue analysis**: Seven critical blocking issues identified across NCCL, UCX, and vLLM, with proposed solutions for each.

2. **Adaptive middleware**: A WFA-based tensor classifier with dual-path routing that differentiates latency-sensitive from throughput-sensitive transfers.

3. **RDMA Tensor Cache**: A precision management layer in Neumann that resolves heterogeneous GPU precision mismatches through FP32 master weight storage and stochastic rounding.

4. **Pluggable transport**: Support for TCP, Soft-RoCE, hardware RoCE, and TTPoe through a unified abstraction, with integration into both vLLM and exo frameworks.

5. **Experimental validation framework**: Three-pillar methodology targeting precision overhead, prefetch effectiveness, and quantization quality, with concrete performance targets.

The work demonstrates that intelligent communication scheduling—treating tensor transfers as first-class objects with metadata-driven routing—can achieve latency improvements on consumer hardware that previously required expensive InfiniBand deployments.

References

[1] L. Ackroyd. Neumann: A unified tensor-based runtime. https://github.com/Shadylukin/Neumann, 2025.

[2] AMD. ROCm vLLM mixture-of-experts guide. https://rocm.blogs.amd.com/software-tools-optimization/vllm-moe-guide/README.html, 2024.

[3] Anthropic. Golden gate claude. https://www.anthropic.com/news/golden-gate-claude, May 2024.

[4] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

[5] R. Chab, S. Setia, and F. Li. Algorithmic techniques for GPU scheduling: A comprehensive survey. *Preprints*, 2025:2025050152, 2025. doi: 10.20944/preprints202505.0152.v1. George Mason University.

[6] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[7] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[9] DeepSeek AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[10] exo explore. exo: Run your own AI cluster at home with everyday devices. https://github.com/exo-explore/exo, 2025.

[11] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[12] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 485–500, 2019.

[13] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[14] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[16] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[17] Linux RDMA. rdma-core documentation. https://github.com/linux-rdma/rdma-core, 2024.

[18] David Louapre. The eiffel tower Llama: Recreating golden gate claude with open-source SAE steering. https://huggingface.co/spaces/dlouapre/eiffel-tower-llama, 2025.

[19] K. Mahajan, A. Balasubramanian, A. Singhvi, S. Venkataraman, A. Akella, A. Phanishayee, and S. Chawla. Themis: Fair and efficient GPU cluster scheduling. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 289–304, 2020.

[20] H. Mao, M. Alizadeh, I. Menache, and S. Kandula. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, pages 50–56, 2016.

[21] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of ACM SIGCOMM*, 2019.

[22] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed precision training. *International Conference on Learning Representations (ICLR)*, 2018.

[23] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[24] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 481–498, 2020.

[25] NVIDIA. GDRCopy: A low-latency GPU memory copy library. https://github.com/NVIDIA/gdrcopy, 2024.

[26] NVIDIA. HPC-X v2.15 documentation. https://docs.nvidia.com/networking/display/hpcxv215, 2024.

[27] NVIDIA. NCCL: NVIDIA collective communications library. https://github.com/NVIDIA/nccl, 2024.

[28] NVIDIA. NIXL: NVIDIA inference xfer library. https://github.com/NVIDIA/nixl, 2025.

[29] Open Compute Project. OCP falcon networking specification. https://github.com/opencomputeproject/OCP-NET-Falcon, 2025.

[30] OpenAI. gpt-oss-120b model card. https://huggingface.co/openai/gpt-oss-120b, 2025.

[31] OpenUCX Consortium. UCX: Unified communication X. https://github.com/openucx/ucx, 2024.

[32] D. Patel and K. Chen. NVIDIA tensor core evolution: From volta to blackwell. SemiAnalysis, June 2025. Accessed February 2026.

[33] B. Peng et al. Hetis: Heterogeneous GPU-cluster scheduling for efficient large language model fine-tuning. *Proceedings of the 2024 USENIX Annual Technical Conference*, 2024.

[34] J. Pennington, R. Socher, and C. D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[35] G. F. Pfister. An introduction to the InfiniBand architecture. *High Performance Mass Storage and Parallel I/O*, 2001.

[36] A. Qiao, S. K. Choe, S. J. Subramanya, W. Neiswanger, Q. Ho, H. Zhang, G. R. Ganger, and E. P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In

Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2021.

[37] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

[38] Red Hat. GPUDirect RDMA on OpenShift AI: Enabling remote GPU memory access for accelerated computing. https://www.redhat.com/en/blog/gpudirect-rdma-openshift-ai-enabling-remote-gpu-memory-access-for-accelerated-computing, 2024.

[39] B. Rothenberger, K. Taranov, A. Perrig, and T. Hoefler. ReDMArk: Bypassing RDMA security mechanisms. In *Proceedings of the 30th USENIX Security Symposium*, 2021.

[40] B. D. Rouhani et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[41] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.

[42] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2020.

[43] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[44] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024.

[45] Tesla. TTPoE at hot chips 2024: Replacing TCP for low latency applications. Hot Chips 2024, 2024.

[46] Tesla, Inc. TTPoe: Time-triggered protocol over ethernet. https://github.com/teslamotors/ttpoe, 2024.

[47] Ultra Ethernet Consortium. Ultra Ethernet Consortium 1.0 specification. https://ultraethernet.org/, 2025.

[48] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[49] vLLM. Issue #22279: FlashAttention 3 fails on blackwell GPUs. https://github.com/vllm-project/vllm/issues/22279, 2026.

[50] vLLM Contributors. RFC: UCC integration for collective communication (PR #30930). https://github.com/vllm-project/vllm/pull/30930, 2025.

[51] J. Xing, K. Hsu, Y. Qiu, Z. Yang, H. Liu, and A. Chen. Bedrock: Programmable network support for secure RDMA systems. In *Proceedings of the 31st USENIX Security Symposium*, 2022.

[52] Z. Zhong et al. Efficient quantum circuit scheduling with batch-buffered overlap and dependency-aware contraction. *arXiv preprint arXiv:2509.04955*, 2025.

A Scripts and Configurations

All implementation code, deployment configurations, and benchmark scripts are provided in the `scripts/` directory alongside this document:

File	Type	Purpose
`wfa_classifier.py`	Python	WFA tensor classifier and phase detection
`dual_path_rdma.py`	Python	Hot/cold QP pool routing
`rdma_buffer_pool.py`	Python	NUMA-aware pyverbs buffer pool
`transports.py`	Python	TransportBase, TCP/RoCE/TTPoe, Selector, NCCLConfig
`exo_rdma_integration.py`	Python	RDMADiscovery, RDMATransport, TopologyManager
`vllm_connector.py`	Python	AdaptiveRDMAConnector, InstrumentedKVCacheManager
`benchmark_vllm.py`	Python	gpt-oss-120b inference benchmark
`benchmark_transports.py`	Python	Multi-transport comparison with LaTeX output
`benchmark_all.sh`	Bash	Automated benchmark across all transports
`start_cluster.sh`	Bash	Unified cluster start with transport selection
`load_ttpoe.sh`	Bash	TTPoe kernel module loader
`docker-compose.head.yaml`	YAML	Ray head node + vLLM server deployment
`docker-compose.worker.yaml`	YAML	Ray worker node deployment

Table 22: Scripts and configuration files provided with this report

B Expected Transport Performance

Based on prior testing (Section 11.1) and transport specifications:

Transport	Latency	Expected Throughput	CPU Overhead	Status
TCP	∼1 ms	Baseline	High	Always available
Soft-RoCE	∼190 μs	+10–15%	Medium	Deprecated
Hardware RoCE	<2 μs	+15–25%	Low	Requires Mellanox
Tesla TTPoe	∼2 μs	+20–30%	Near-zero	Requires module

Table 23: Expected transport performance for distributed LLM inference

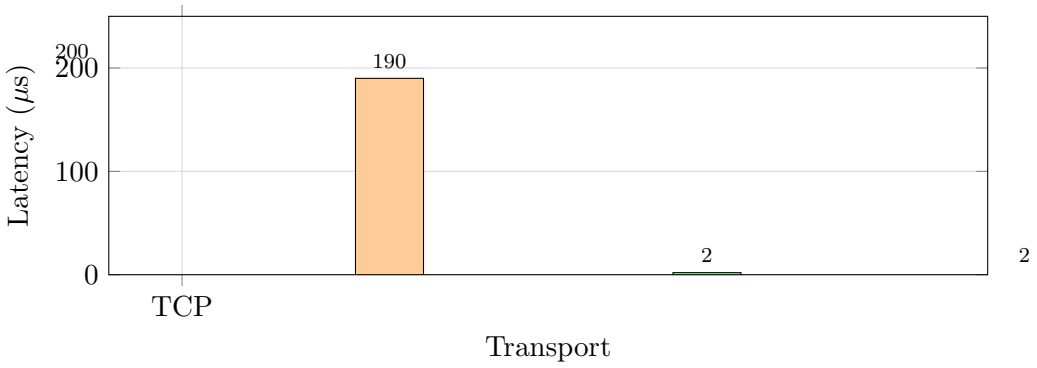

Figure 16: Transport latency comparison—TCP baseline vs RDMA alternatives. Hardware RoCE and TTPoe achieve 100× lower latency than software solutions.